UNIVERSIDAD TECNOLÓGICA DE PANAMÁ

FACULTAD DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

CAMPUS VÍCTOR LEVIS SASSO

DESARROLLO WEB VII: PHP Y MYSQL

INFORME TÉCNICO DE LABORATORIO

Seguridad en PHP con OpenSSL

Programación segura según OWASP – Protege los datos

Estudiante	Angel Gálvez
Cédula	8-1038-857
Correo	angel.galvez2@utp.ac.pa
Asignatura	Desarrollo Web VII: PHP y MySQL
Grupo	IS3122
Docente	Ing. Irina Fong
Período	II Semestre 2026
Fecha de entrega	30 de octubre de 2026
Repositorio	github.com/galvezangel074-glitch/OpenSSL

Contenido

1. Introducción al tema
2. Objetivo del laboratorio
3. Entorno y estructura del proyecto
4. Capturas de pantalla con descripciones
5. Documentación técnica de funciones
6. Investigación y ejemplo práctico con Libsodium (Sodium)
7. Dificultades encontradas y soluciones
8. Conclusiones personales
9. Referencias

1. Introducción al tema

La criptografía aplicada permite que un sistema web proteja la información que maneja aunque un atacante llegue a tocar el servidor, la base de datos o la red. La guía del laboratorio parte de la categoría **A02:2021 – Cryptographic Failures** del OWASP Top 10: los datos sensibles (contraseñas, tokens, tarjetas, registros de salud, secretos del negocio) **nunca deben almacenarse ni viajar en texto plano**, y lo mejor es evitar guardar datos sensibles cuando no es necesario.

OpenSSL es una biblioteca de código abierto que implementa los protocolos TLS/SSL y ofrece funciones para generar llaves, administrar certificados X.509, cifrar y descifrar, y firmar datos. PHP la expone mediante la extensión `openssl` (funciones `openssl_*`). Con ella se resuelven los dos pilares que pide el laboratorio:

Pilar	Qué garantiza	Técnicas usadas en el laboratorio
Confidencialidad	Solo quien posee la llave puede leer el dato, en reposo y en tránsito.	Cifrado simétrico (AES-128-CBC, AES-256-CBC, AES-256-GCM, XSalsa20-Poly1305) y asimétrico (RSA-OAEP, sealed box X25519).
Integridad y autenticidad	El dato no fue alterado por terceros y proviene de quien dice haberlo enviado.	Firmas digitales (RSA con SHA-256, Ed25519), HMAC-SHA3-512 y etiquetas de autenticación AEAD (GCM, Poly1305).

La guía recomienda además revisar **Libsodium (Sodium)**, incluida por defecto en PHP desde la versión 7.2, que ofrece criptografía moderna pensada para ser segura por defecto. El laboratorio ejecuta los scripts entregados por la docente, les agrega formularios, documenta cada función y complementa el trabajo con una investigación y un ejemplo ejecutable de Sodium.

2. Objetivo del laboratorio

Objetivo general

Aplicar la extensión OpenSSL y librerías criptográficas modernas (Libsodium) en PHP, comprendiendo su propósito para proteger datos sensibles según los lineamientos de OWASP, mediante la ejecución de ejemplos prácticos y el análisis de su documentación técnica.

Objetivos específicos y metas alcanzadas

Meta	Resultado
Verificar y habilitar OpenSSL y Sodium en WampServer	Ambas extensiones activas (OpenSSL 3.0.18, libsodium 1.0.18). Figura 2.
Ejecutar localmente los scripts entregados	Los 10 scripts funcionan; se corrigieron la ruta de `openssl.cnf` y el uso del CSR (sección 7).
Agregar formularios para recibir los datos (E.2.2)	Todos los scripts reciben sus datos por formulario con token CSRF y salida escapada.
Probar `firma7`, `firmaOtra`, `FirmarMensaje` y la encriptación simple (E.2.3)	Probados con entrada válida y con datos alterados; evidencias en la sección 4.
Documentar las funciones criptográficas	Tablas con nombre, propósito, parámetros, valor devuelto y proceso (sección 5).
Investigar e implementar Libsodium	Investigación teórica y cinco demostraciones ejecutables (sección 6).

3. Entorno y estructura del proyecto

3.1 Entorno utilizado

Componente	Versión / detalle
Sistema operativo	Windows 11 Home
Servidor local	WampServer 64 bits · Apache 2.4.65
PHP	8.3.28 (extensiones `openssl` y `sodium` habilitadas)
OpenSSL	3.0.18 (30 Sep 2025)
Libsodium	1.0.18
Editor y control de versiones	Visual Studio Code · Git y GitHub

3.2 Estructura de archivos

Archivo	Función	Tipo
`Informacion.php`	Estado de OpenSSL y Sodium, algoritmos disponibles	Verificación
`Encriptacion.php`	Encriptación simple: AES-128-CBC, AES-256-CBC, AES-256-GCM	Requerido
`firma7.php`	Par de llaves RSA, firma y verificación	Requerido
`firmaOtra.php`	Llave privada, CSR y certificado X.509 autofirmado	Requerido
`FirmarMensaje.php`	Firma con la llave de firmaOtra y verificación con su certificado	Requerido
`sodium.php`	Libsodium: secretbox, Ed25519, Argon2id, sealed box, rendimiento	Sección 1.6
`encriptacionOpenssl_2.php`	AES-256-CBC + HMAC-SHA3-512 (Encrypt-then-MAC)	Complementario
`firmaDiprena.php`	Firma de un trámite de personal (sin base de datos)	Complementario
`firma4.php`	Solicitud de certificado (CSR)	Complementario
`firmaDigital.php`	Cifrado RSA-OAEP con llaves exportadas a archivo	Complementario
`includes/bootstrap.php`	Configuración común: localizar `openssl.cnf`, CSRF, escape XSS	Soporte
`originales/`	Scripts entregados por la docente, sin modificar	Referencia

3.3 Medidas de seguridad aplicadas (OWASP)

- **CSRF:** cada formulario incluye un token de sesión que se valida en el servidor.
- **XSS:** toda salida se escapa con `htmlspecialchars` (función `h()`).
- **Llaves privadas:** se generan en tiempo de ejecución, están en `.gitignore` y sus carpetas tienen `.htaccess` con `Require all denied`.
- **Claves derivadas:** las contraseñas de usuario se convierten en llaves con PBKDF2-SHA256 (OpenSSL) o Argon2id (Sodium), con sal aleatoria.
- **Aleatoriedad segura:** IV y nonces generados con `openssl_random_pseudo_bytes` o `random_bytes`.

4. Capturas de pantalla con descripciones

Cada captura se tomó ejecutando el formulario en el navegador (Edge) sobre `http://localhost/OPENSSL/`; el pie de página de cada captura muestra el nombre del estudiante, la versión de PHP, OpenSSL y libsodium, y la fecha de ejecución.

4.1 Inicio y verificación del entorno

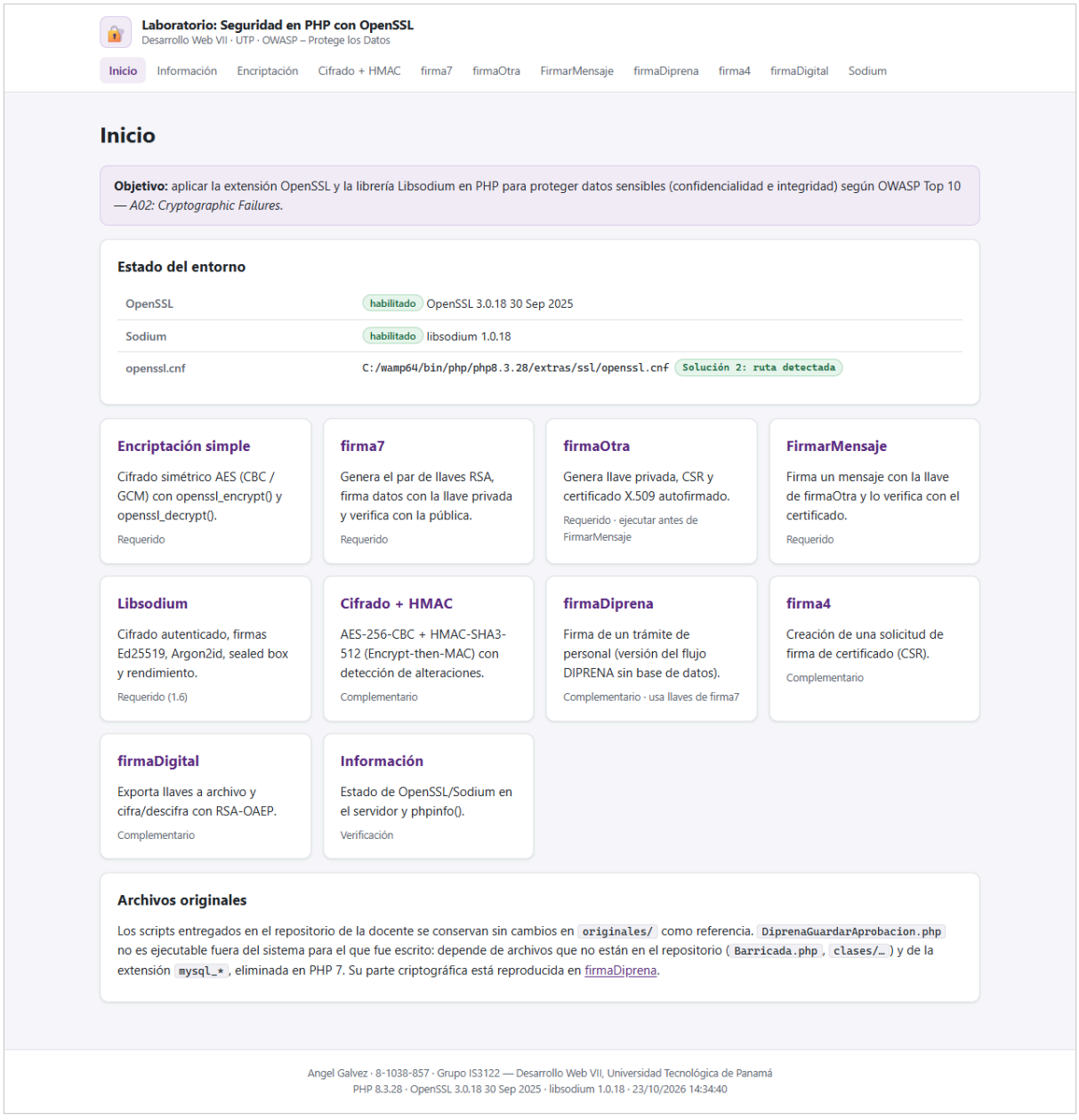


Figura 1. Página de inicio del laboratorio

> Menú con los diez scripts y el estado del entorno: OpenSSL y Sodium habilitados, y la ruta de `openssl.cnf` detectada automáticamente (Solución 2 de la guía).

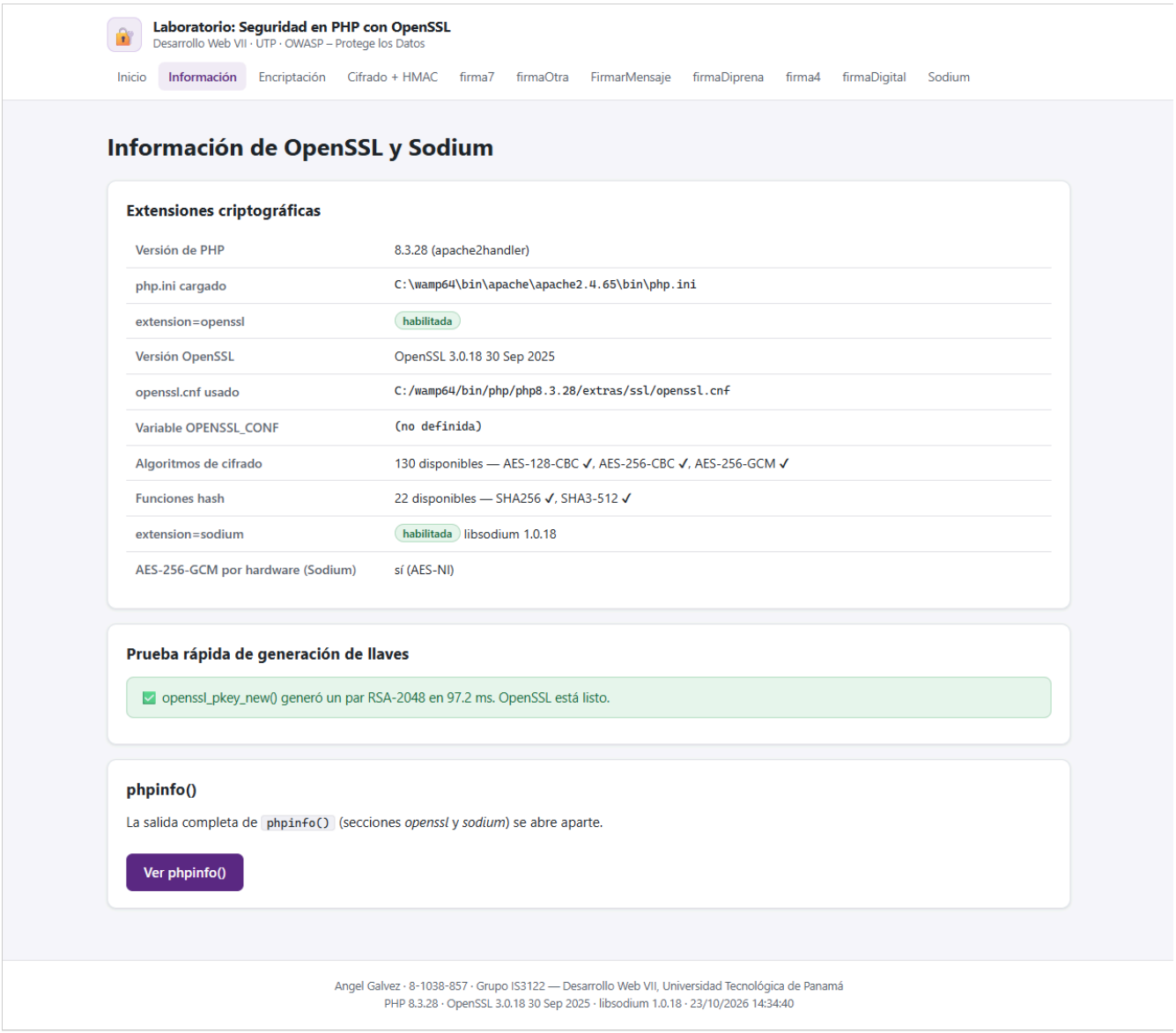


Figura 2. Informacion.php – estado de las extensiones

> Confirma que la extensión OpenSSL está activa, la versión de PHP, el `php.ini` cargado, los algoritmos AES/SHA disponibles y que Sodium está habilitada. Es la verificación pedida en E.1.2.

4.2 Encriptación simple con OpenSSL (Encriptacion.php)

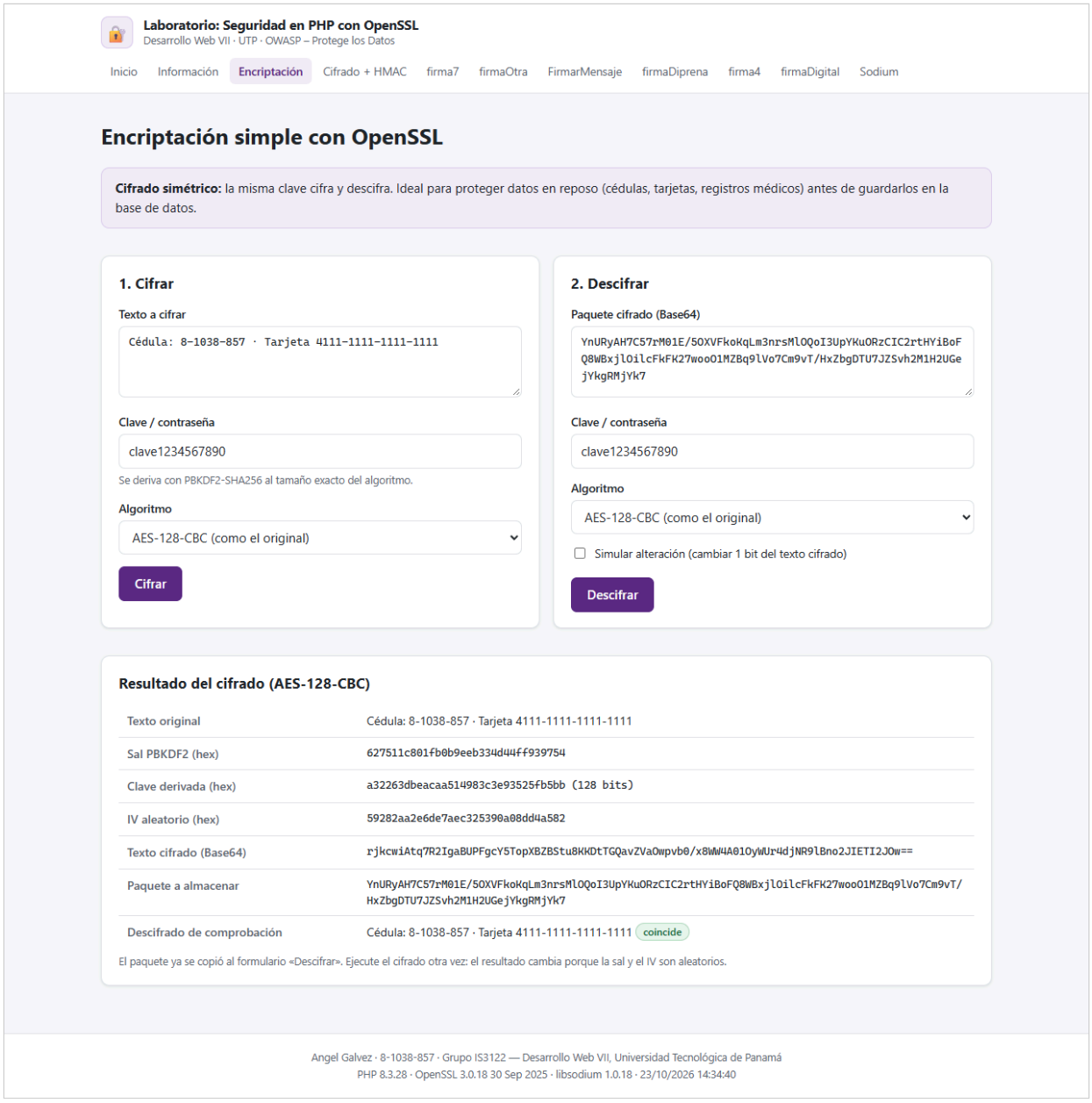


Figura 3. Cifrado con AES-128-CBC

> Entrada: texto y contraseña. El script deriva la llave con PBKDF2, genera un IV aleatorio y muestra el paquete cifrado en Base64 (sal + IV + cifrado). El texto original ya no es legible.

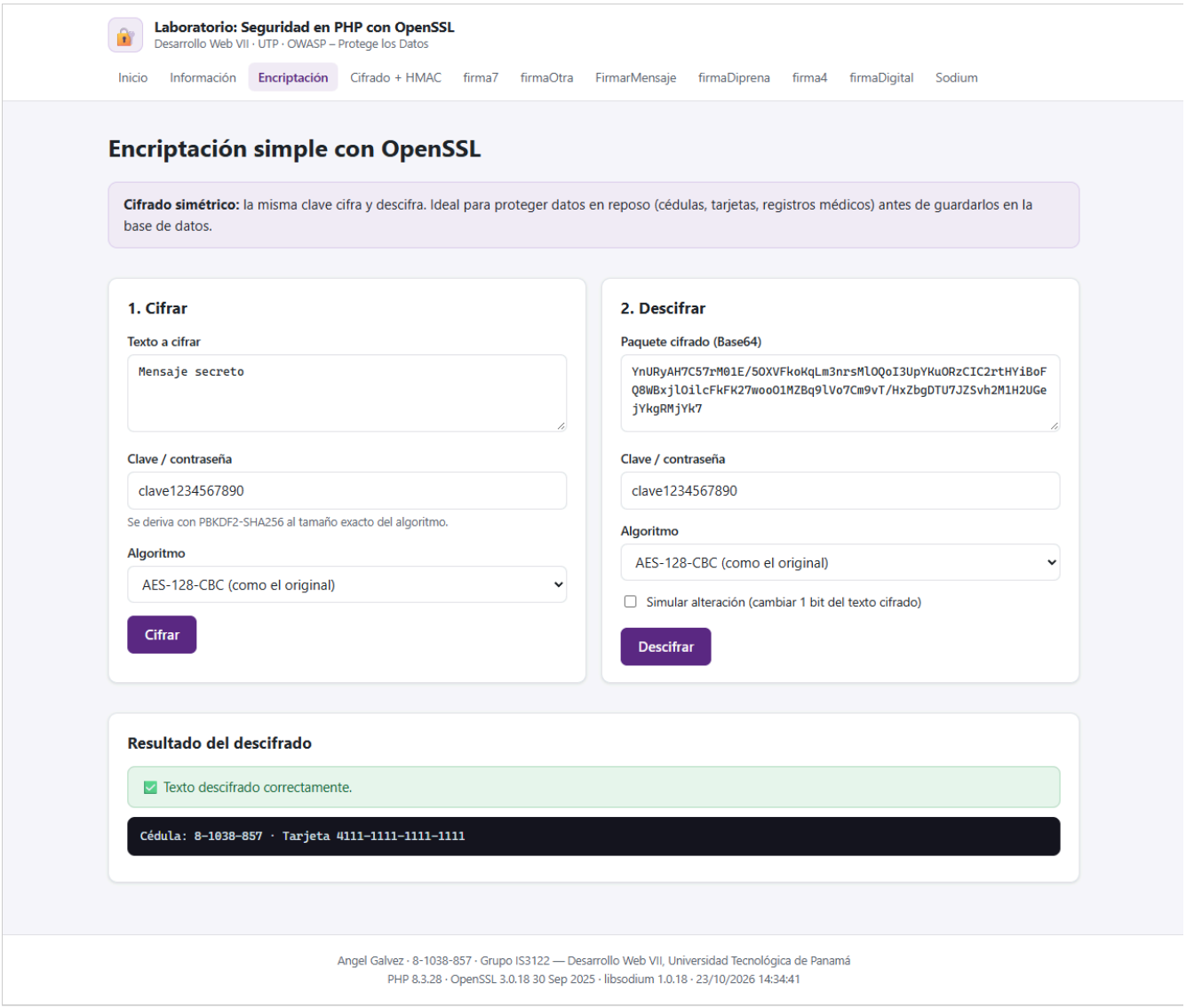


Figura 4. Descifrado del paquete

> Se pega el paquete y la misma contraseña; `openssl_decrypt` recupera el texto original, lo que demuestra que el cifrado es reversible solo con la llave.

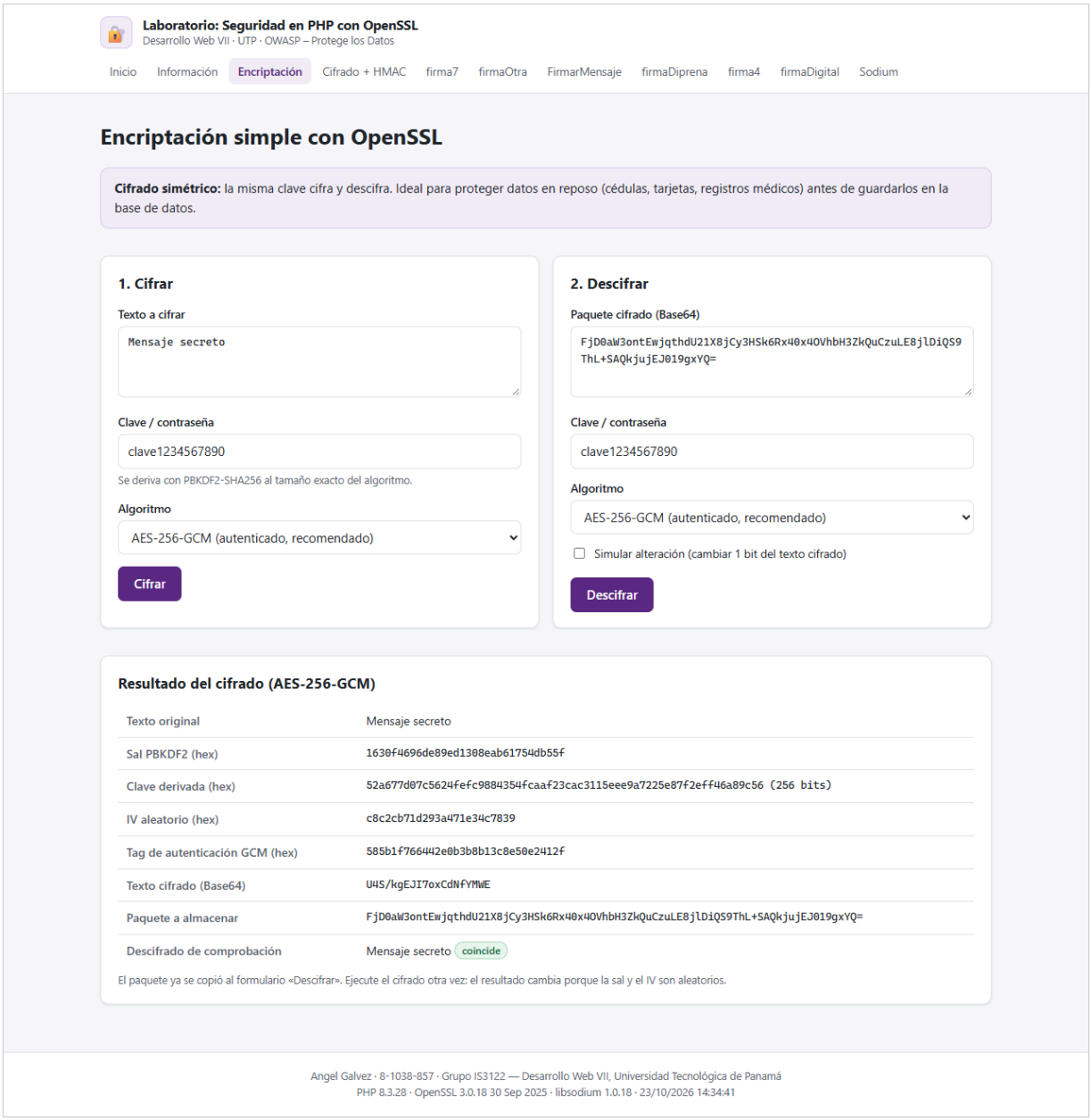


Figura 5. Cifrado con AES-256-GCM

> GCM es un modo autenticado: además del cifrado produce una etiqueta (tag) de 16 bytes que se guarda en el paquete.

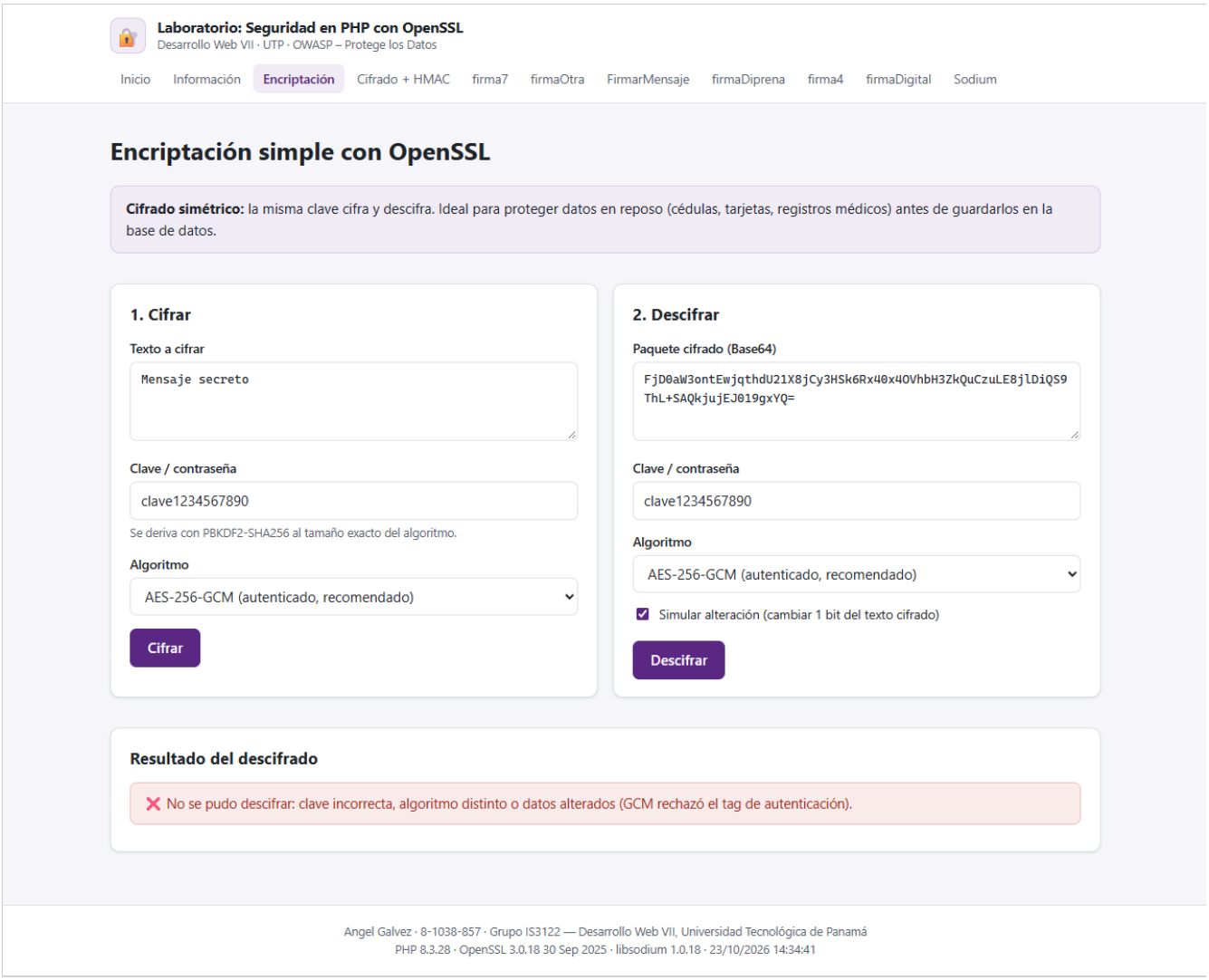


Figura 6. GCM con el paquete alterado

> Al modificar el paquete antes de descifrar, el tag de autenticación no coincide y `openssl_decrypt` rechaza el mensaje. Demuestra la integridad que CBC por sí solo no ofrece.

4.3 Cifrado con HMAC (encriptacionOpenssl_2.php)

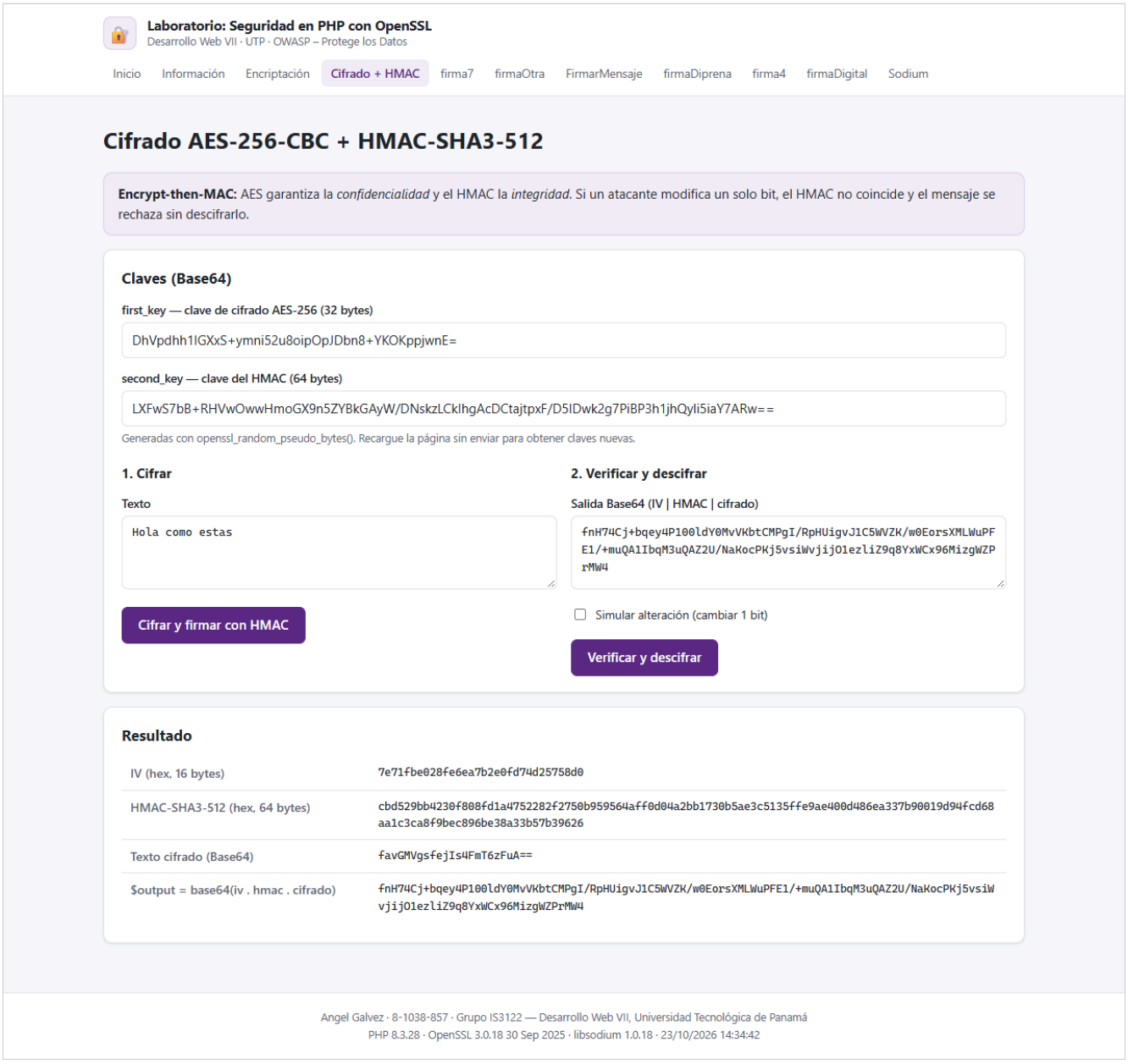


Figura 7. AES-256-CBC + HMAC-SHA3-512 – cifrar

> Dos llaves independientes (cifrado y HMAC). La salida es `base64(iv . hmac . cifrado)`.

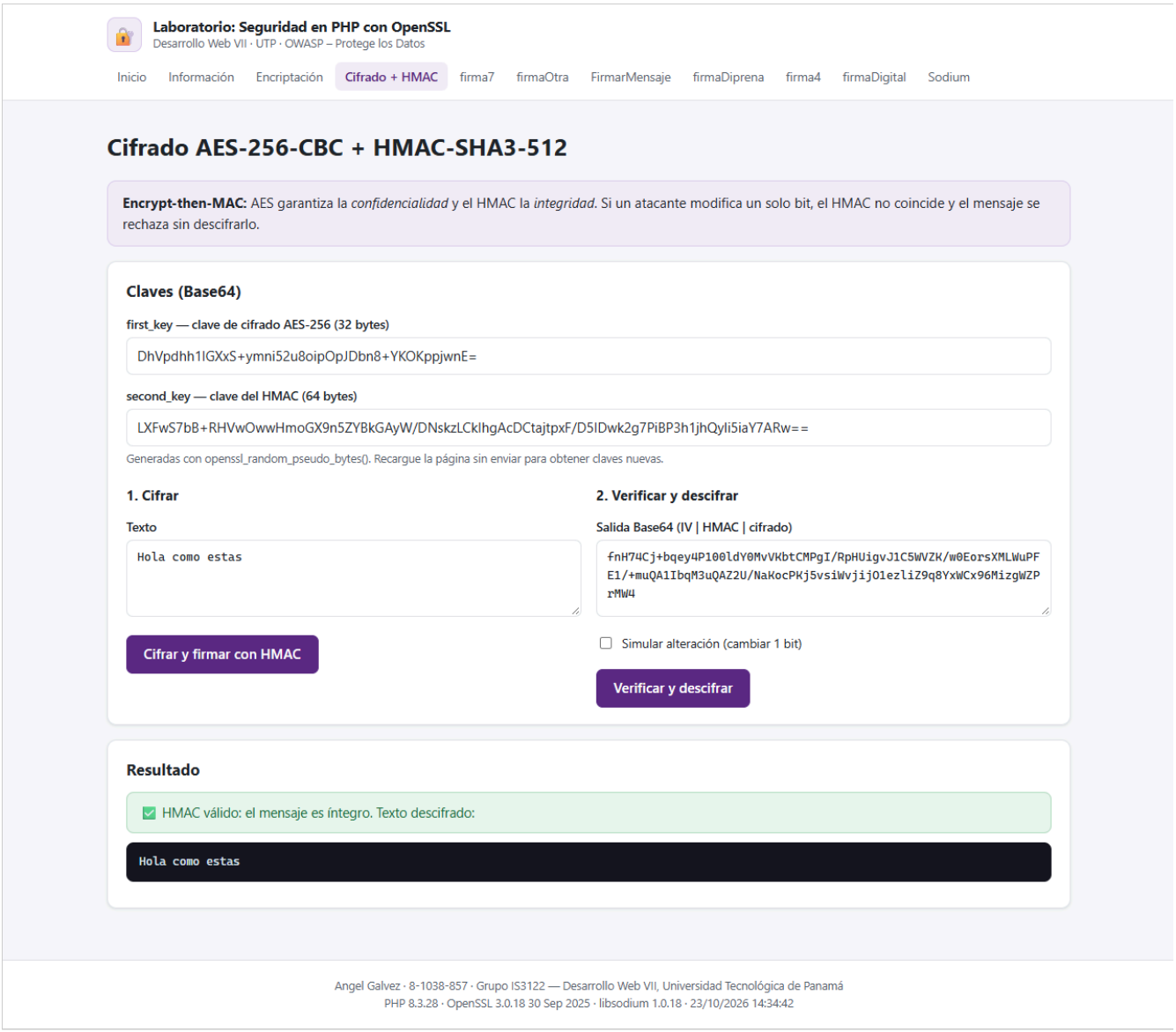


Figura 8. Verificación del HMAC y descifrado

> El HMAC se recalcula y se compara en tiempo constante; si es válido, se descifra el texto.

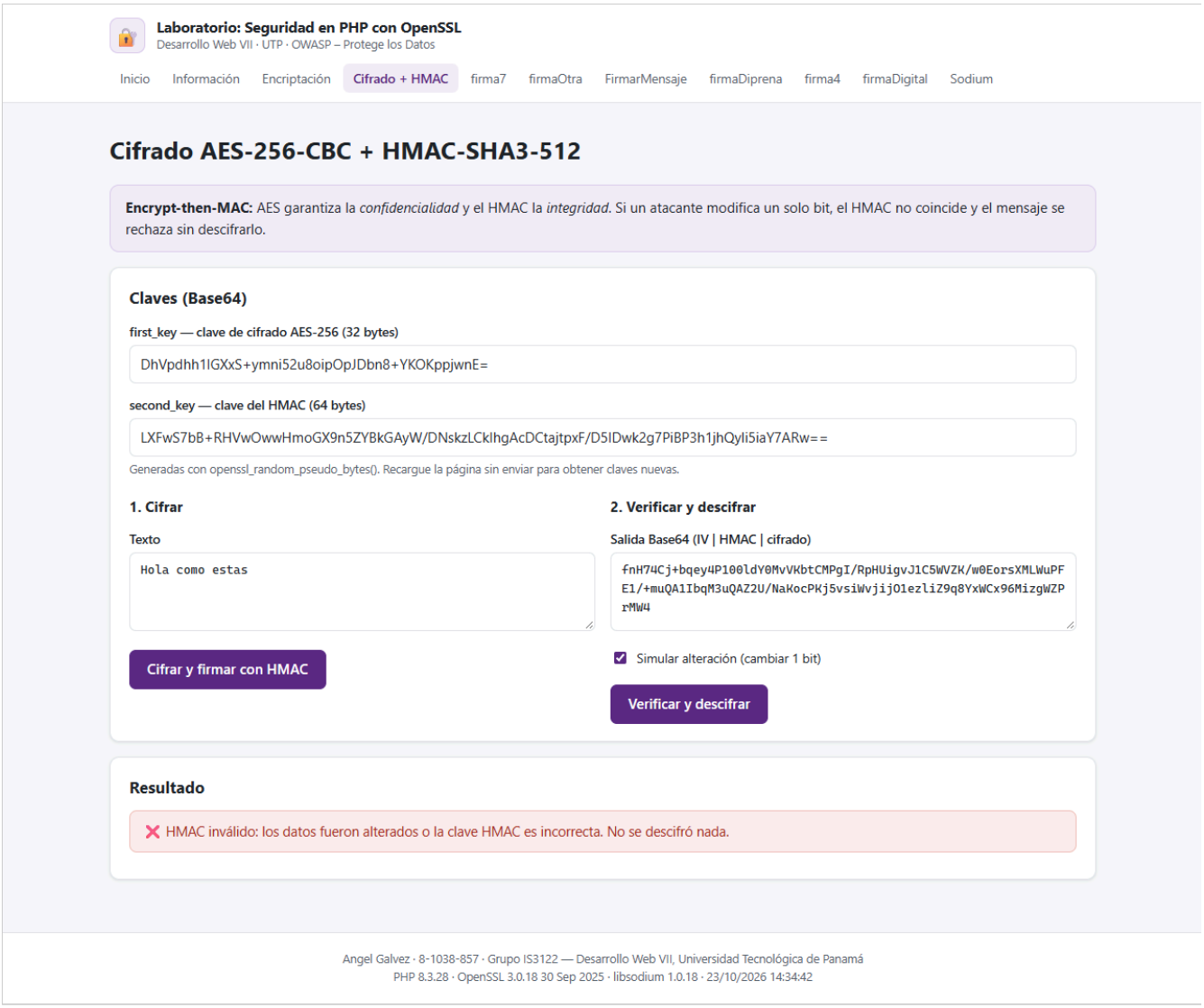


Figura 9. Texto alterado: HMAC inválido

> Un cambio en el cifrado produce un HMAC distinto y el script se niega a descifrar (Encrypt-then-MAC).

4.4 firma7.php – par de llaves RSA y firma digital

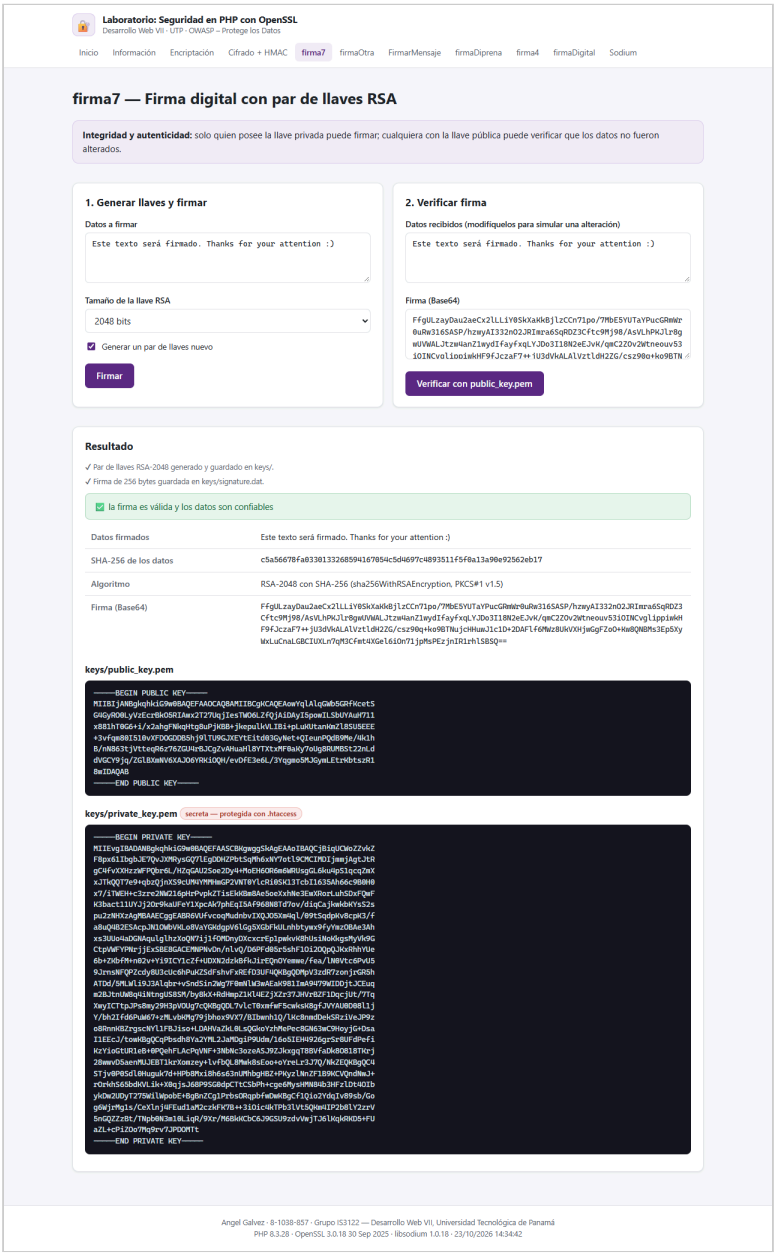


Figura 10. Generación de llaves RSA-2048 y firma

Entrada: datos a firmar y tamaño de llave. Resultado: llaves guardadas en `keys/`, firma de 256 bytes (Base64), huella SHA-256 de los datos y las llaves PEM generadas. Las llaves son de práctica y no se usan en ningún sistema.

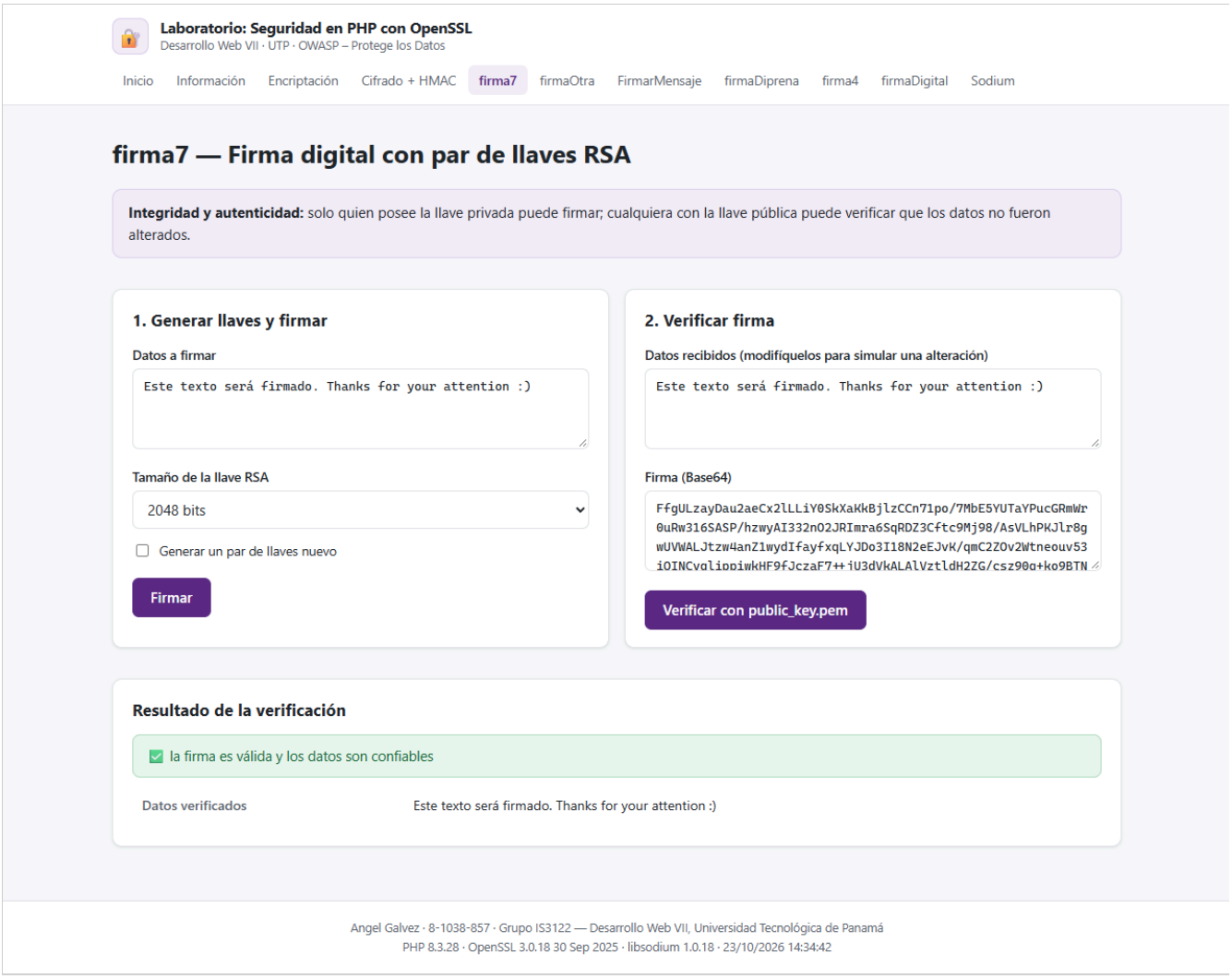


Figura 11. Verificación con los datos originales

> `openssl_verify` devuelve 1: la firma es válida y los datos son confiables.

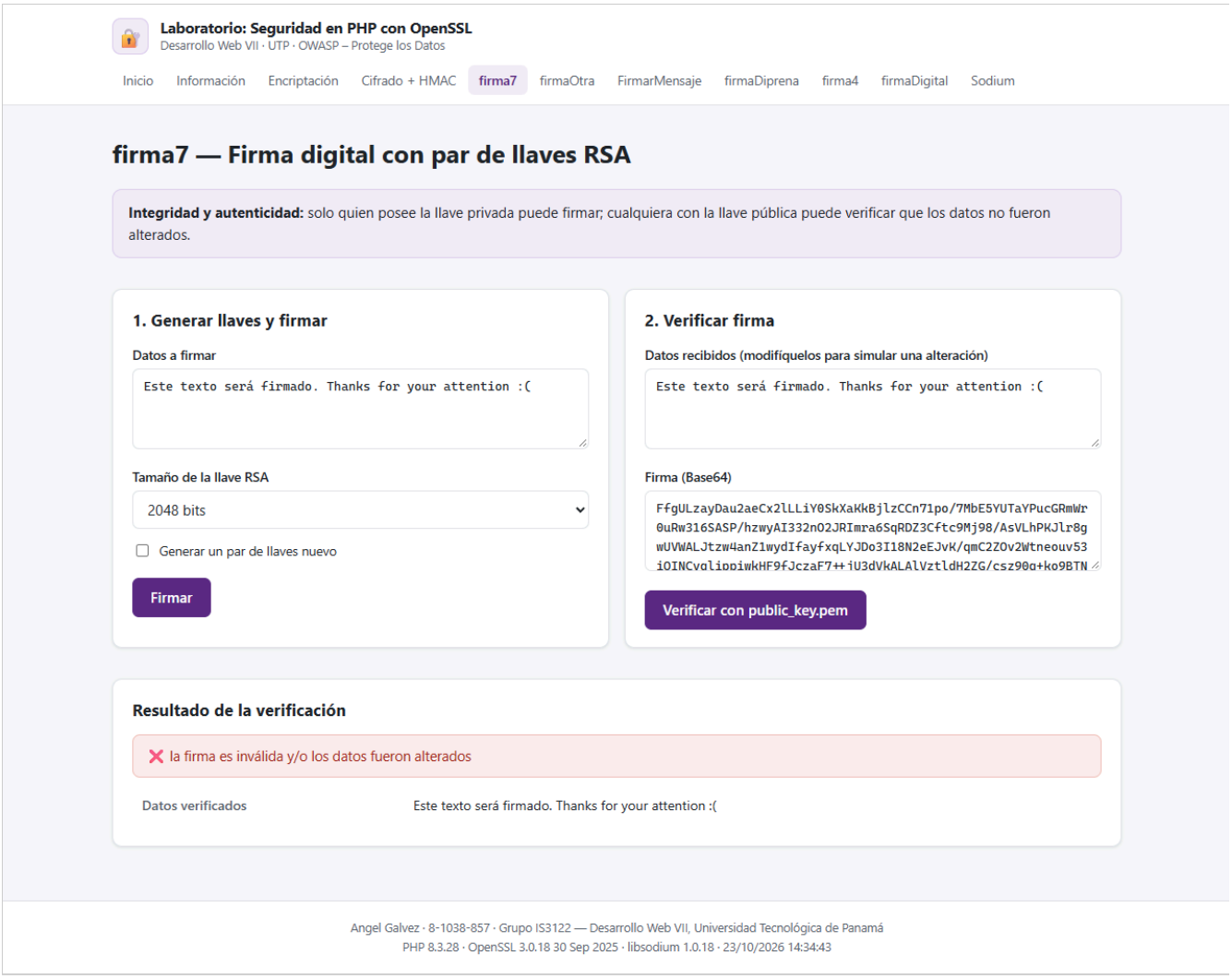


Figura 12. Verificación con los datos alterados

> Cambiar un solo carácter hace que `openssl_verify` devuelva 0: la firma es inválida. Demuestra el principio de integridad.

4.5 firmaOtra.php y FirmarMensaje.php – certificado X.509

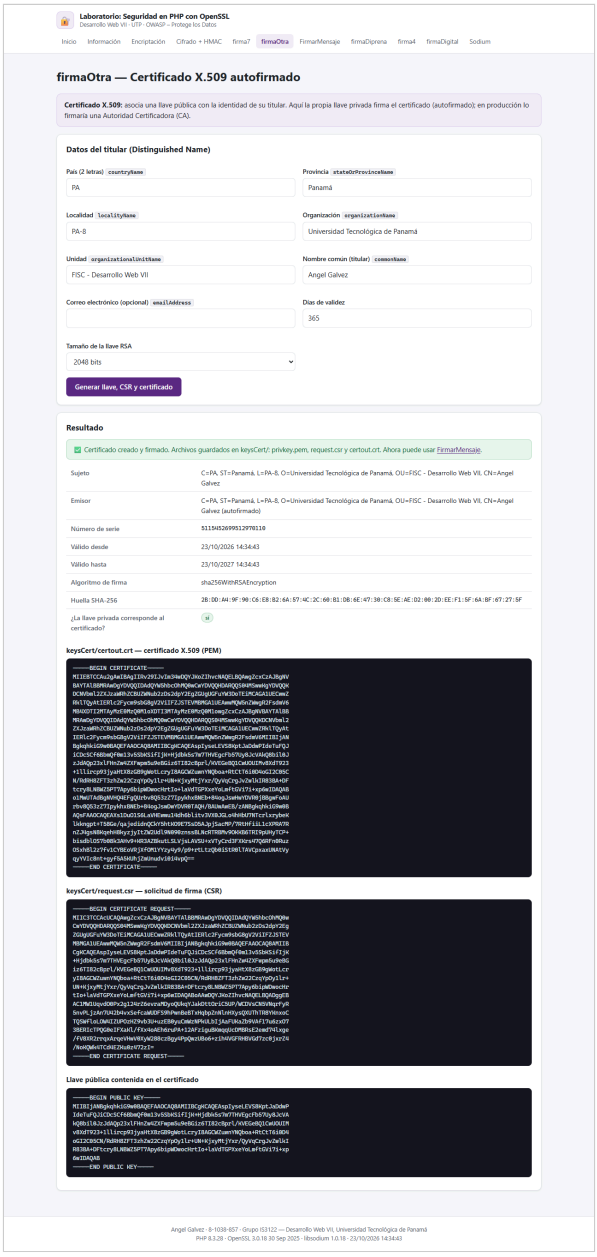


Figura 13. Llave privada, CSR y certificado autofirmado

Entrada: datos del titular (país, provincia, organización, unidad, nombre común, días de validez). Resultado: certificado `certout.crt`, solicitud `request.csr` y llave `privkey.pem`, con sujeto, emisor, número de serie, vigencia (365 días), algoritmo sha256WithRSAEncryption, huella SHA-256 y la comprobación de que la llave privada corresponde al certificado.

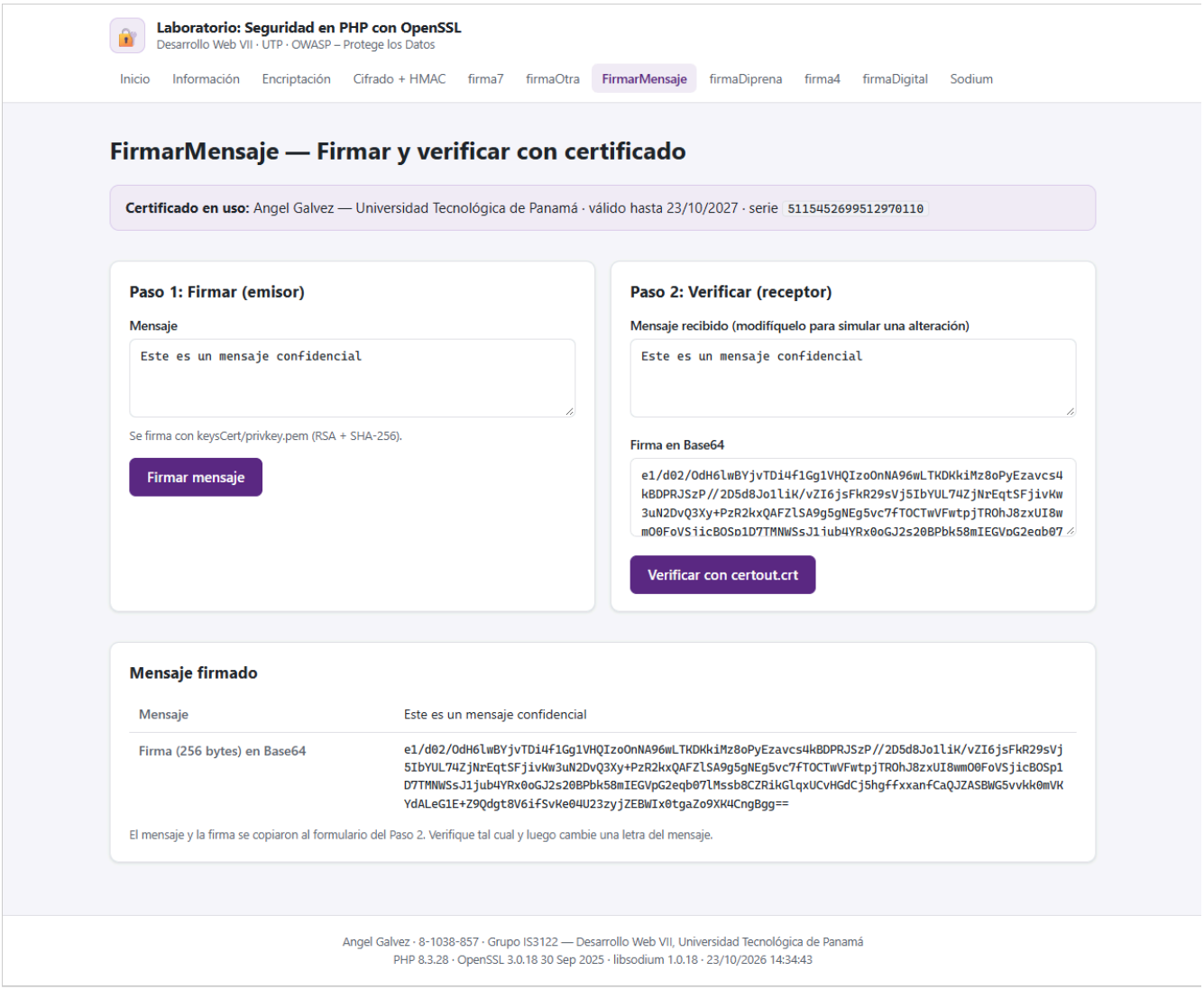


Figura 14. Firma de un mensaje con la llave de firmaOtra

> El mensaje se firma con `keysCert/privkey.pem` y la firma se muestra en Base64.

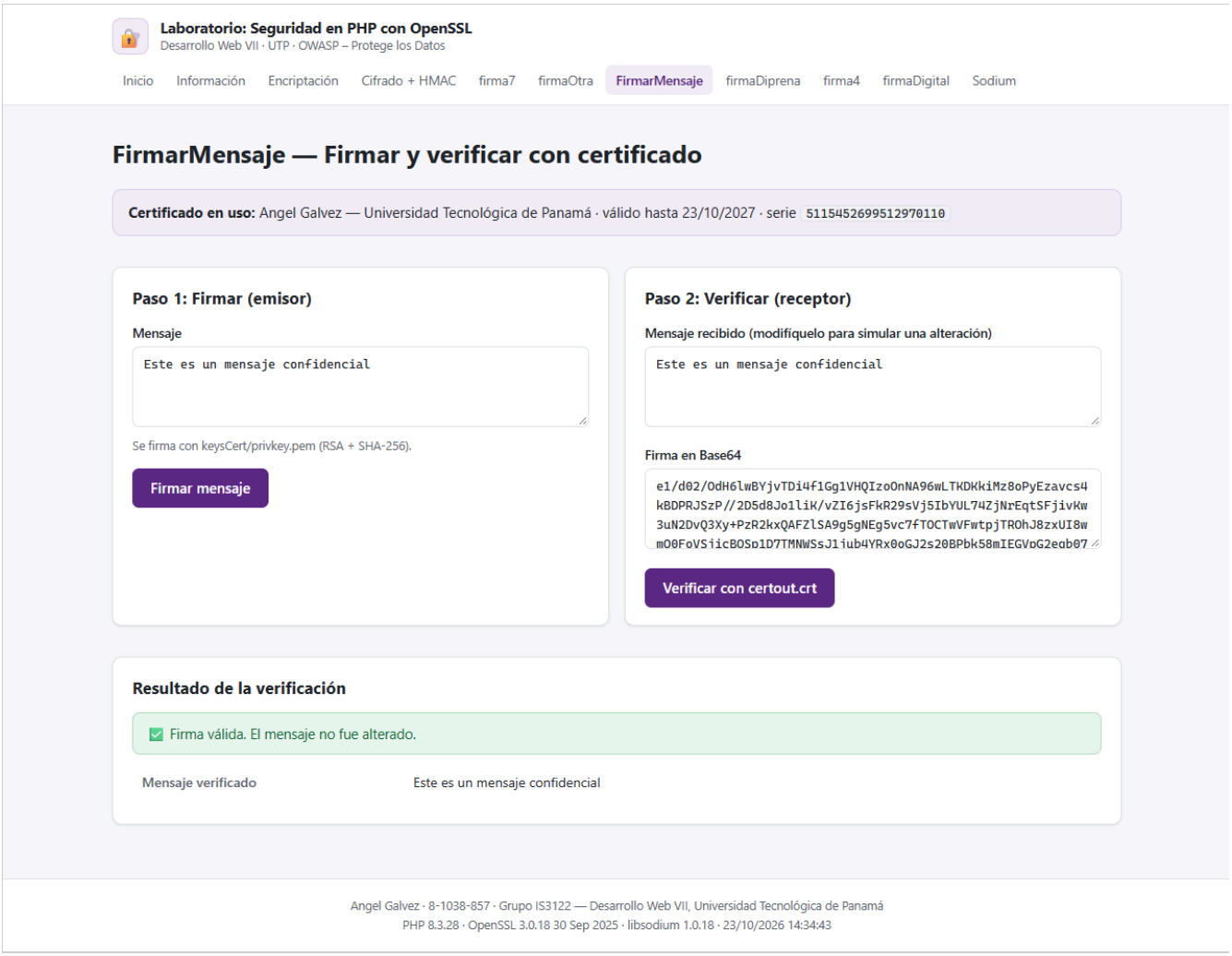


Figura 15. Verificación válida con el certificado

> La llave pública se extrae del certificado y la firma se comprueba: el mensaje no fue alterado.

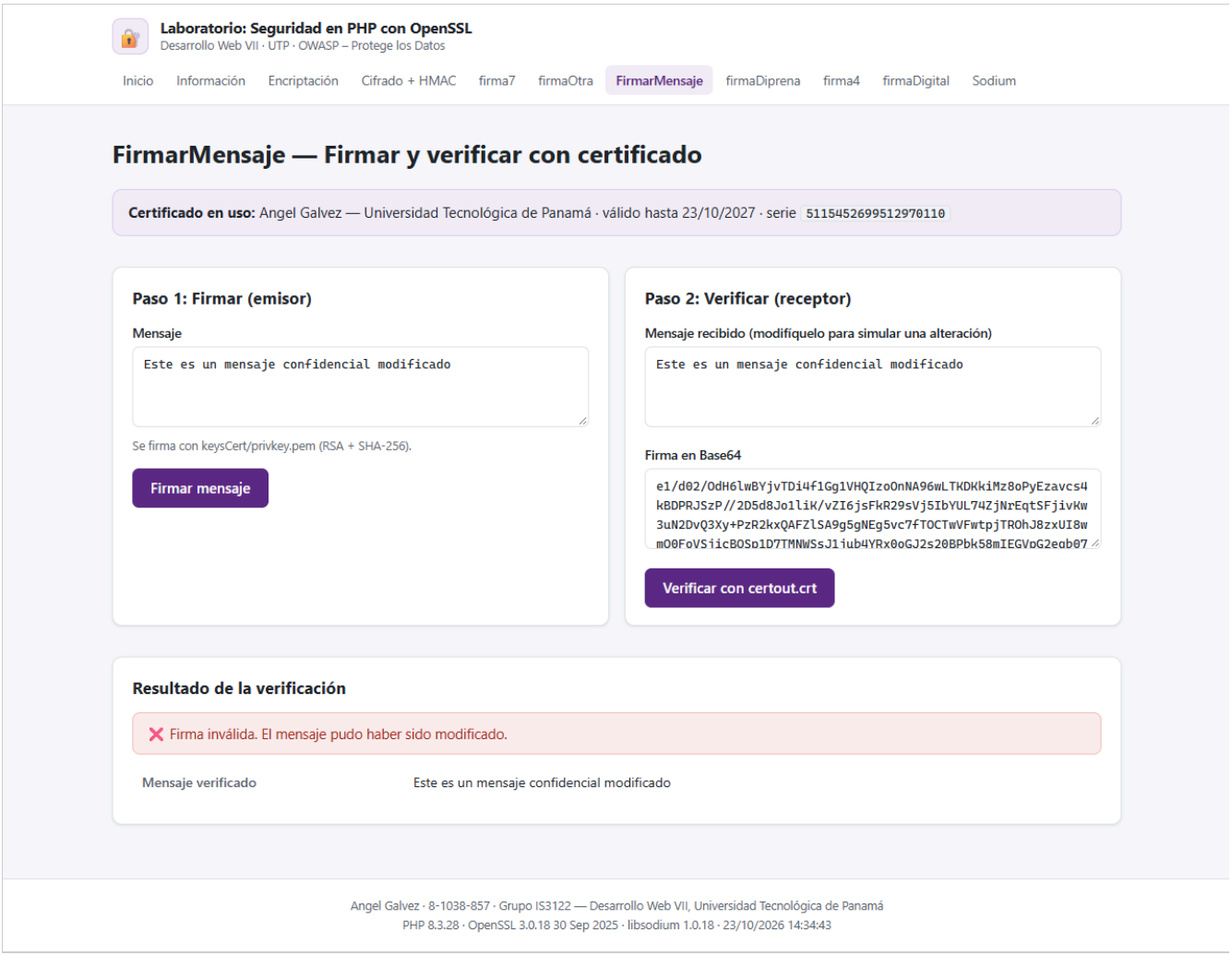


Figura 16. Mensaje alterado: firma inválida

> Con un mensaje distinto al firmado, la verificación falla.

4.6 Scripts complementarios

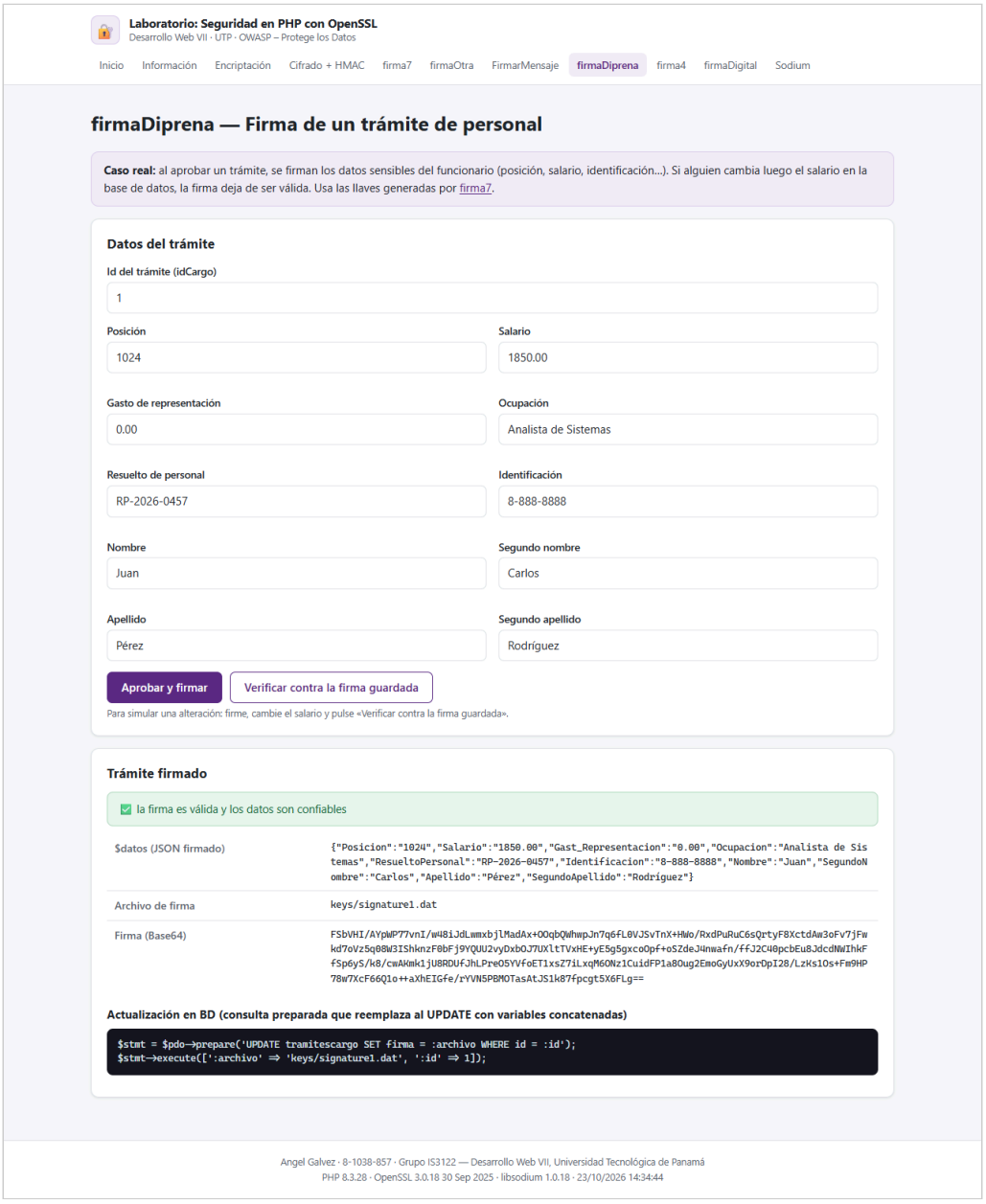


Figura 17. firmaDiprena – firma de un trámite

> Los campos del trámite se serializan con `json_encode` y se firman con las llaves de firma7.

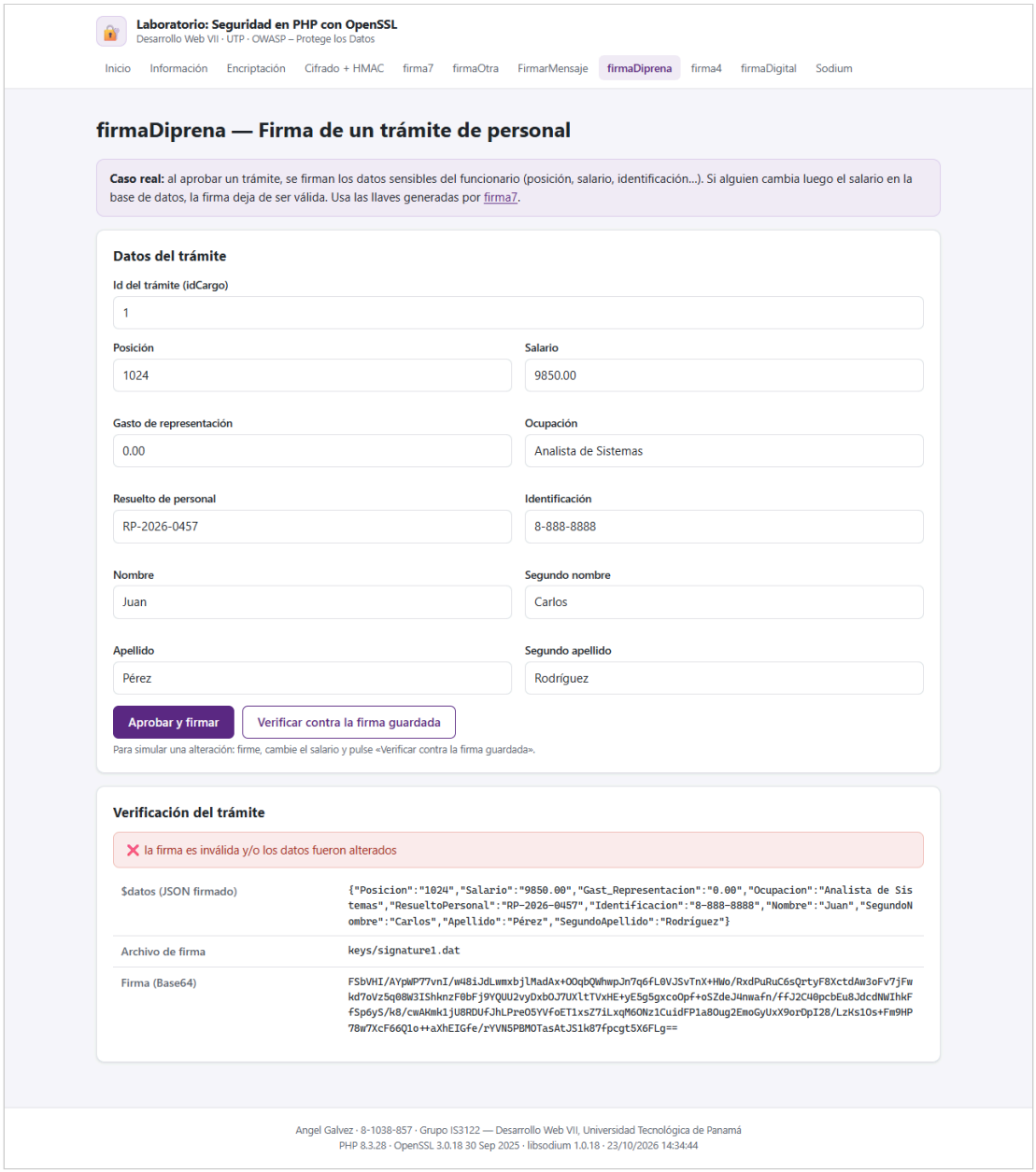


Figura 18. firmaDiprena – salario alterado

> Modificar el salario invalida la firma: la verificación detecta la manipulación del dato sensible.

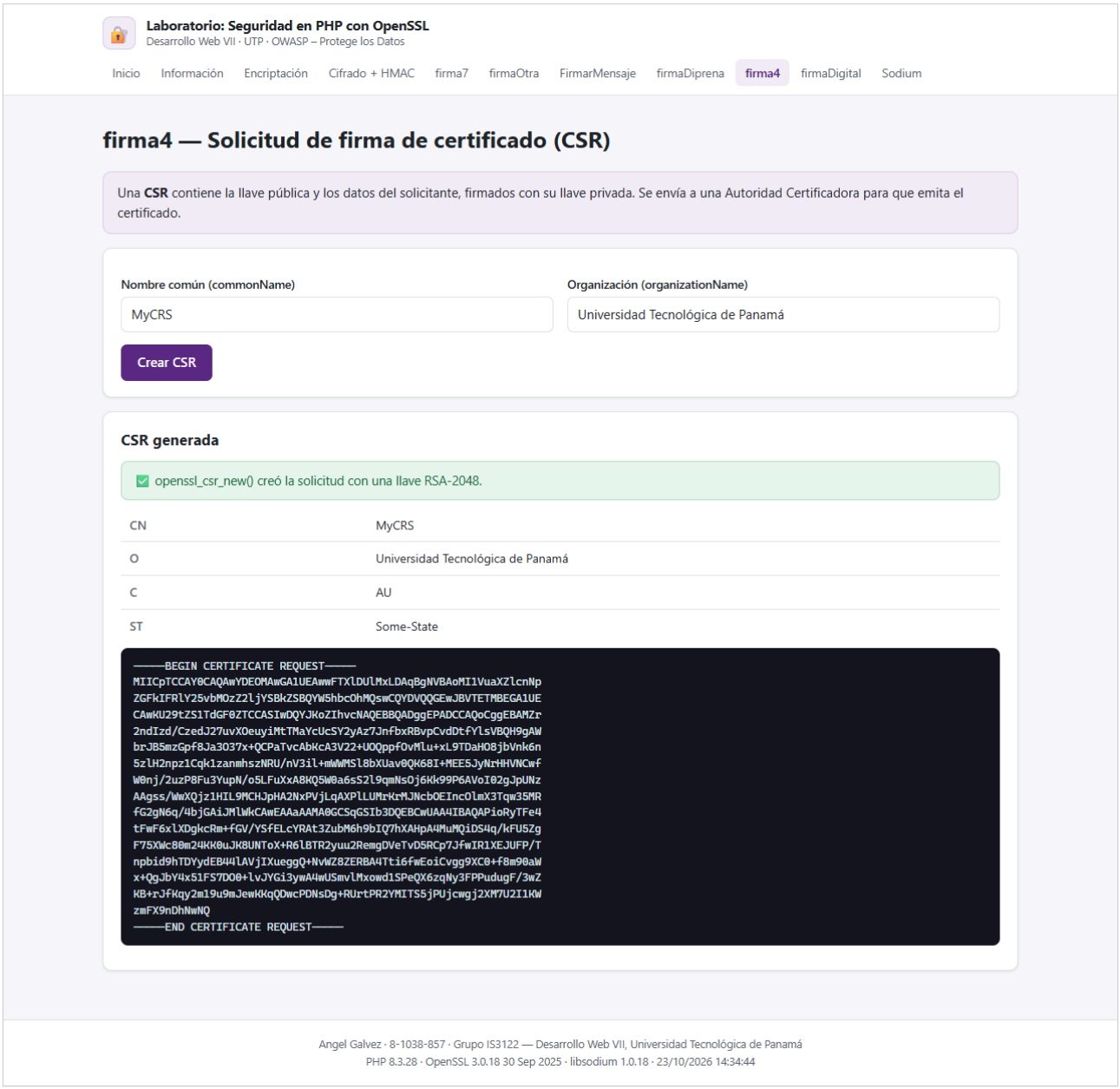


Figura 19. firma4 – solicitud de certificado (CSR)

> Muestra la solicitud generada, su sujeto y la llave pública incluida.

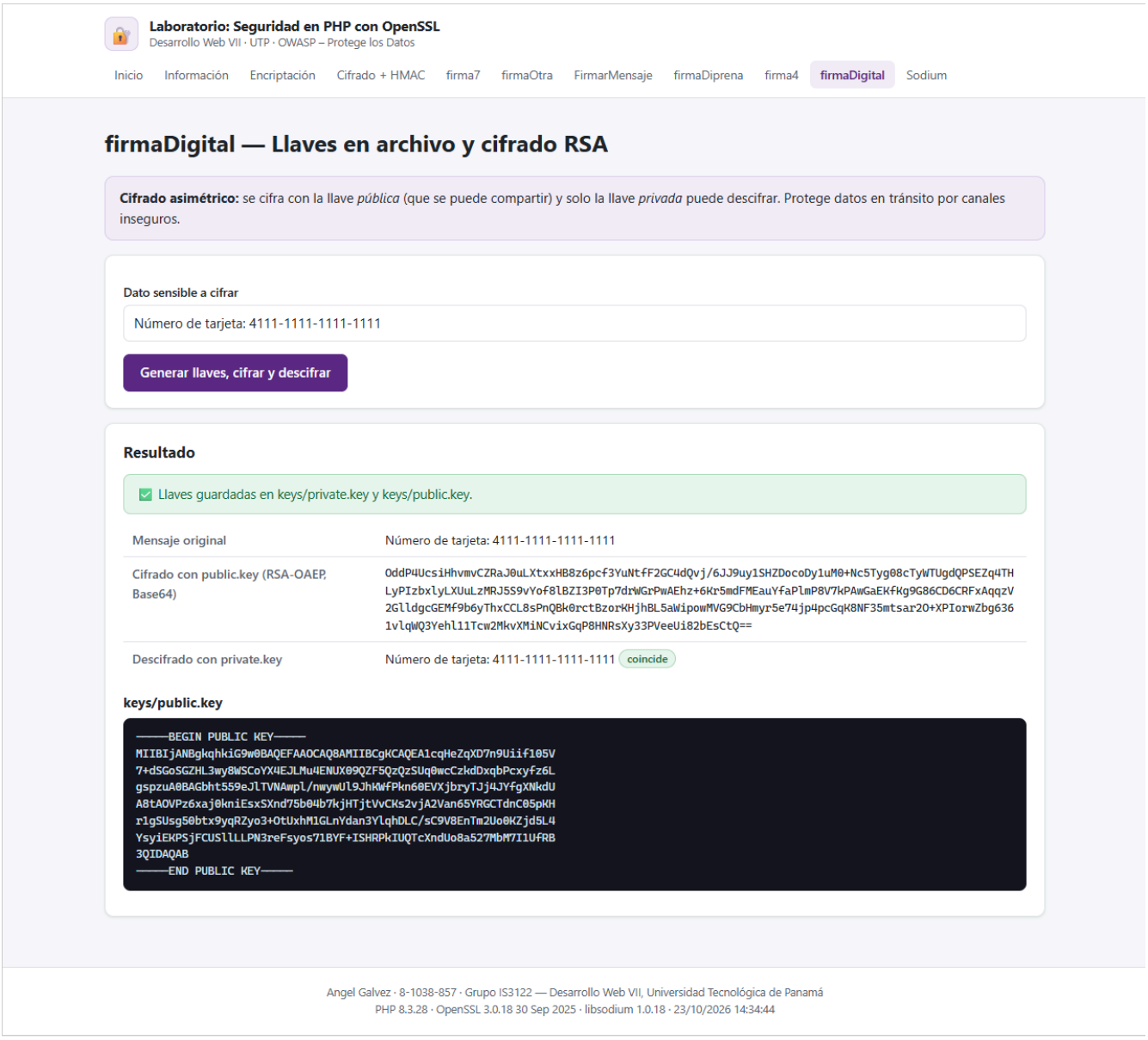


Figura 20. firmaDigital – cifrado RSA-OAEP

> Cifra con la llave pública y descifra con la privada, usando el relleno OAEP.

4.7 Libsodium (sodium.php)

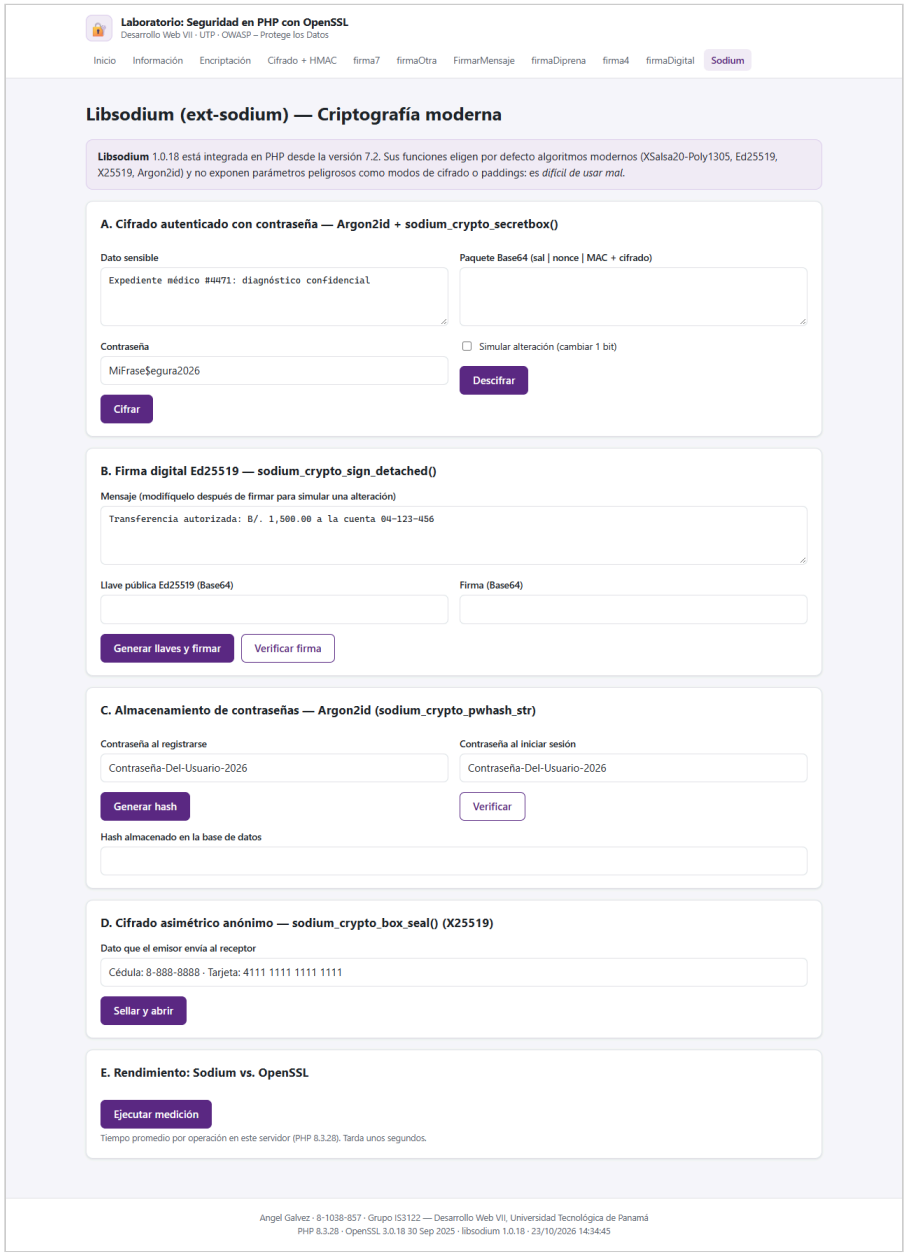


Figura 21. Página de Sodium

> Resumen de las cinco demostraciones y de las versiones de libsodium y PHP.

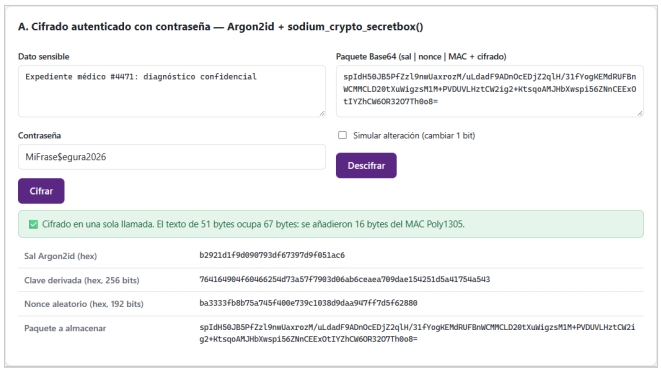


Figura 22. secretbox – cifrar

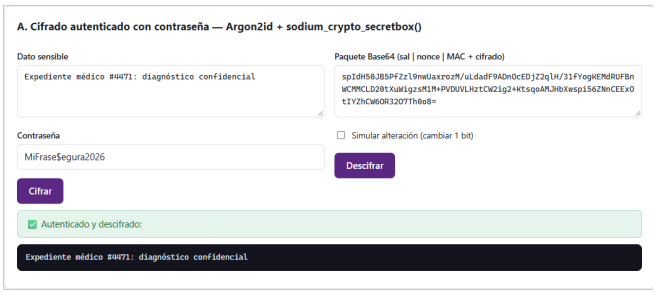


Figura 23. secretbox – descifrar

La llave se deriva de la contraseña con Argon2id; `sodium_crypto_secretbox` cifra y autentica en una sola llamada y el descifrado devuelve el texto original.

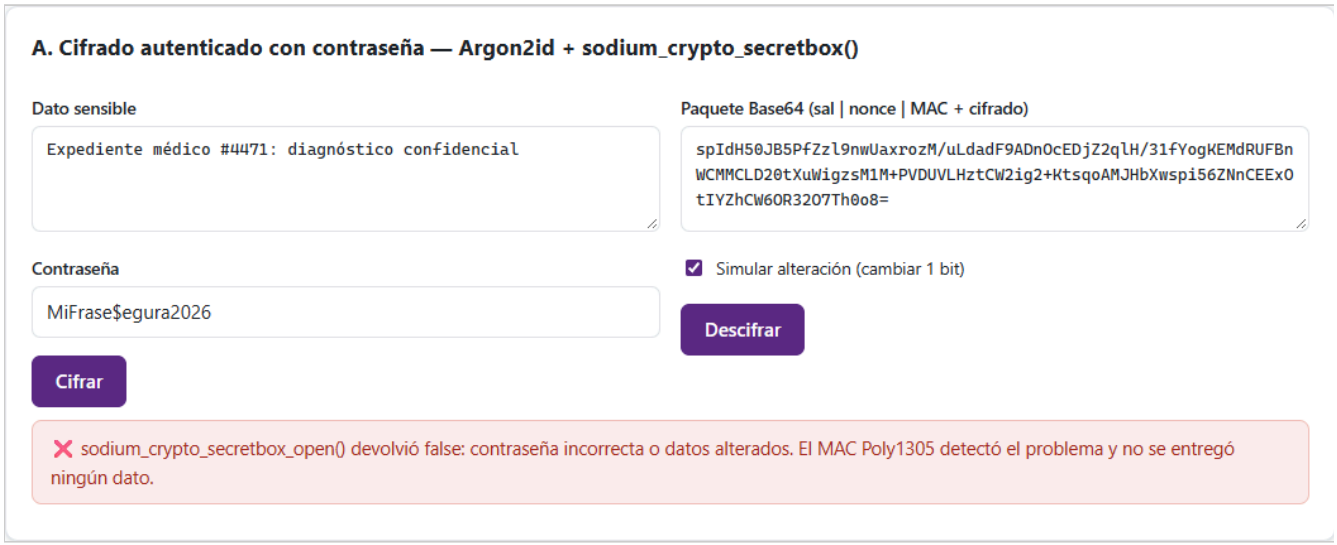


Figura 24. secretbox con el cifrado alterado

`sodium_crypto_secretbox_open` devuelve `false`: el MAC Poly1305 detecta la alteración.



Figura 25. Ed25519 – firmar

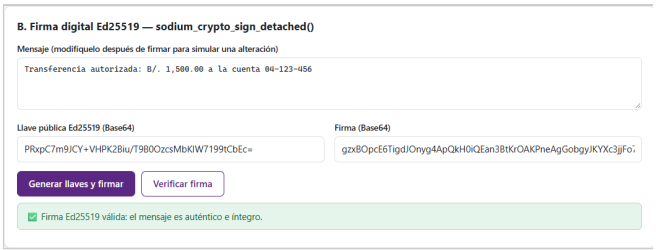


Figura 26. Ed25519 – verificar

Firma separada de 64 bytes con llave pública de 32 bytes.

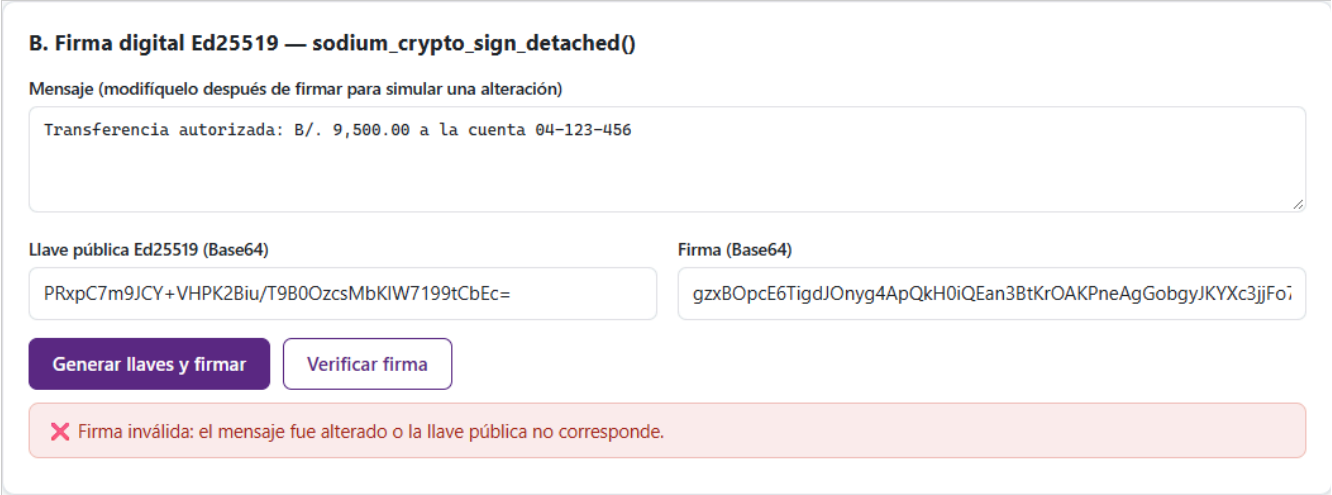


Figura 27. Ed25519 con el mensaje alterado

La verificación falla al cambiar el mensaje.



Figura 28. Argon2id – hash

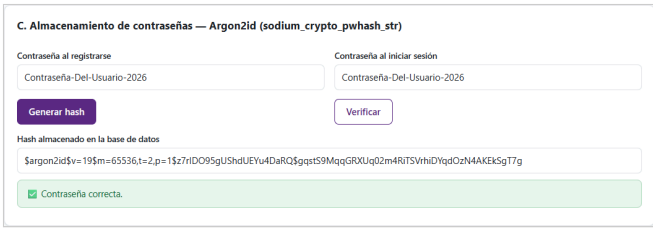


Figura 29. Contraseña correcta

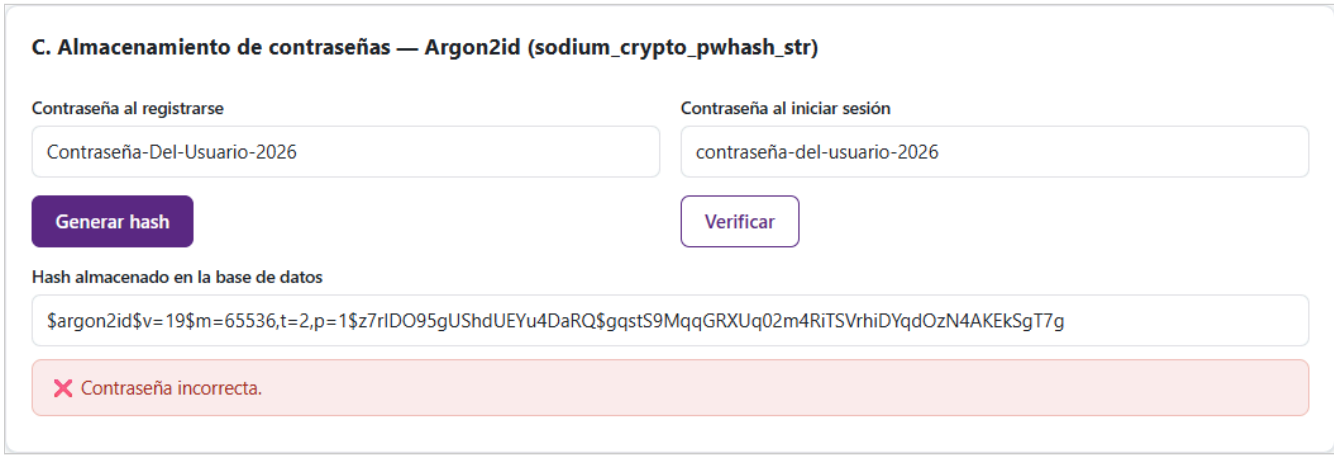


Figura 30. Argon2id – contraseña incorrecta

El hash almacenado incluye algoritmo, parámetros y sal; `sodium_crypto_pwhash_str_verify` acepta solo la contraseña original.



Figura 31. Sealed box (cifrado asimétrico anónimo)

> Cualquiera cifra con la llave pública del destinatario; solo el dueño de la llave privada puede abrir el mensaje.

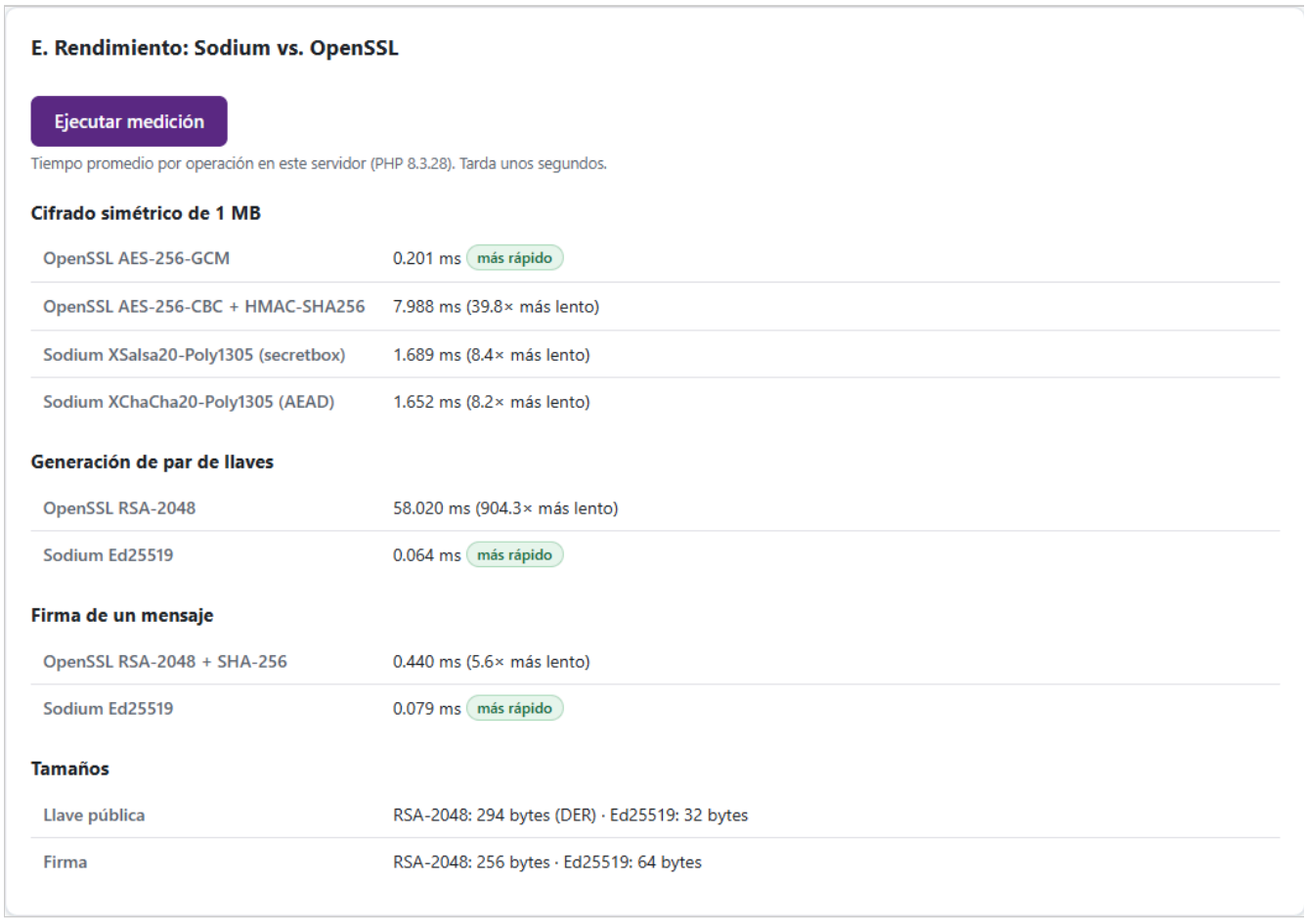


Figura 32. Comparativa de rendimiento OpenSSL vs Sodium

> Medición en este equipo: AES-256-GCM de OpenSSL (con aceleración por hardware) cifra 1 MB en 0.201 ms, frente a 1.689 ms de secretbox; en cambio, Ed25519 genera llaves en 0.064 ms frente a 58.020 ms de RSA-2048 y firma 5.6 veces más rápido.

5. Documentación técnica de funciones

Cada función se documenta con los cinco elementos exigidos: nombre, propósito general, parámetros recibidos, valor devuelto y explicación del proceso criptográfico.

5.1 Scripts del laboratorio (pruebas requeridas)

`firma7.php – firma y verificación RSA`	
Propósito general	Generar un par de llaves RSA, firmar un texto y comprobar su integridad.
Parámetros recibidos	`datos` (texto), `bits` (2048/4096), `regenerar` (checkbox), `firma` (Base64) y `csrf`.
Valor devuelto	Página con la firma en Base64 (guardada en `keys/signature.dat`), SHA-256 de los datos, llaves PEM y el resultado «firma válida / inválida».
Proceso criptográfico aplicado	`openssl_pkey_new` → `openssl_pkey_export` / `openssl_pkey_get_details` → `openssl_sign` (SHA-256 + RSA PKCS#1 v1.5) → `openssl_verify` con la llave pública.

`firmaOtra.php – certificado X.509 autofirmado`	
Propósito general	Crear la identidad digital (llave, CSR y certificado) de una organización.
Parámetros recibidos	Campos del nombre distinguido (país, provincia, localidad, organización, unidad, nombre común, correo), días de validez, bits de la llave y `csrf`.
Valor devuelto	`keysCert/privkey.pem`, `request.csr`, `certout.crt` y el detalle del certificado (sujeto, emisor, serie, vigencia, huella).
Proceso criptográfico aplicado	`openssl_pkey_new` → `openssl_csr_new` → `openssl_csr_sign` (autofirmado) → `openssl_x509_export`; `openssl_x509_check_private_key` confirma que la llave corresponde al certificado.

`FirmarMensaje.php – firma con certificado`	
Propósito general	Firmar un mensaje con la llave de firmaOtra y verificarlo con el certificado.
Parámetros recibidos	`mensaje`, `firma` (Base64) y `csrf`.
Valor devuelto	Firma en Base64 y resultado de `openssl_verify` (1, 0 o -1).
Proceso criptográfico aplicado	`openssl_pkey_get_private` → `openssl_sign` (SHA-256) → `base64_encode`. Para verificar: `base64_decode` → `openssl_pkey_get_public` (desde el certificado) → `openssl_verify`.

`Encriptacion.php – cifrado simétrico`	
Propósito general	Proteger un texto en reposo con AES y recuperarlo con la contraseña.
Parámetros recibidos	`texto`, `clave` (contraseña), `algoritmo` (AES-128-CBC, AES-256-CBC, AES-256-GCM), `paquete` (para descifrar) y `csrf`.
Valor devuelto	Paquete Base64 (sal ‖ IV ‖ tag [solo GCM] ‖ cifrado) y, al descifrar, el texto original o el error.
Proceso criptográfico aplicado	`derivarClave()` aplica PBKDF2-SHA256 con sal aleatoria; el IV se genera con `openssl_random_pseudo_bytes`; después `openssl_encrypt` / `openssl_decrypt`. En GCM, el tag detecta cualquier alteración.

5.2 Scripts complementarios

`encriptacionOpenssl_2.php – AES-256-CBC + HMAC`	
Propósito general	Cifrar con confidencialidad e integridad (Encrypt-then-MAC).
Parámetros recibidos	Texto, `first_key` (cifrado), `second_key` (HMAC), `salida` y `csrf`.
Valor devuelto	Cadena Base64 `iv ‖ hmac ‖ cifrado`; al descifrar, el texto o «HMAC inválido».
Proceso criptográfico aplicado	Cifra con `openssl_encrypt` (AES-256-CBC), calcula `hash_hmac(sha3-512)` sobre el cifrado y lo compara con `hash_equals` antes de descifrar.

`firmaDiprena.php – firma de un trámite`	
Propósito general	Firmar y verificar los datos de un trámite de personal.
Parámetros recibidos	`idCargo` (id del trámite), los diez campos del trámite (posición, salario, gasto de representación, ocupación, resuelto, identificación, nombres y apellidos), `accion` (firmar / verificar) y `csrf`.
Valor devuelto	Firma Base64 guardada para el trámite y resultado de la verificación contra ella.
Proceso criptográfico aplicado	Serializa los campos con `json_encode` (evita ambigüedad entre campos concatenados), firma con `openssl_sign` (SHA-256) y verifica con `openssl_verify`.

`firma4.php – solicitud de certificado (CSR)`	
Propósito general	Generar una CSR con una llave nueva.
Parámetros recibidos	`commonName` (obligatorio), `organizationName` y `csrf`.
Valor devuelto	CSR en PEM, su sujeto y el tamaño de la llave pública incluida.
Proceso criptográfico aplicado	`openssl_pkey_new` → `openssl_csr_new` → `openssl_csr_export`; `openssl_csr_get_subject` y `openssl_csr_get_public_key` para inspeccionarla.

`firmaDigital.php – RSA-OAEP`	
Propósito general	Cifrar con la llave pública y descifrar con la privada.
Parámetros recibidos	`mensaje` y `csrf`.
Valor devuelto	Llave pública, mensaje cifrado en Base64 y el texto descifrado con la llave privada para comprobar el ciclo completo.
Proceso criptográfico aplicado	`openssl_pkey_export_to_file` guarda las llaves; `openssl_public_encrypt` y `openssl_private_decrypt` con `OPENSSL_PKCS1_OAEP_PADDING`.

5.3 Funciones de la extensión OpenSSL

`openssl_pkey_new($config)`	
Propósito general	Generar un par de llaves.
Parámetros recibidos	`array $config`: `private_key_bits`, `private_key_type` (`OPENSSL_KEYTYPE_RSA`), `config` (ruta a openssl.cnf).
Valor devuelto	`OpenSSLAsymmetricKey` o `false`.
Proceso criptográfico aplicado	Generación de dos primos grandes y cálculo del par RSA (módulo, exponente público y privado).

`openssl_pkey_export($key, &$salida, $clave, $config)`	
Propósito general	Exportar la llave privada a PEM.
Parámetros recibidos	Llave, variable de salida por referencia, contraseña opcional y configuración.
Valor devuelto	`bool`.
Proceso criptográfico aplicado	Codifica la llave en PKCS#8/Base64; con contraseña, cifra el PEM.

`openssl_pkey_get_details($key)`	
Propósito general	Obtener datos de una llave, incluida la pública.
Parámetros recibidos	Recurso de llave.
Valor devuelto	`array` con `bits`, `key` (PEM público) y `type`.
Proceso criptográfico aplicado	Extrae el módulo y el exponente público de la llave.

`openssl_sign($datos, &$firma, $llavePrivada, $algoritmo)`	
Propósito general	Firmar datos.
Parámetros recibidos	Datos, firma (por referencia), llave privada, `OPENSSL_ALGO_SHA256`.
Valor devuelto	`bool`; la firma queda en `$firma`.
Proceso criptográfico aplicado	Calcula SHA-256 de los datos y cifra el resumen con la llave privada RSA (PKCS#1 v1.5).

`openssl_verify($datos, $firma, $llavePublica, $algoritmo)`	
Propósito general	Verificar una firma digital.
Parámetros recibidos	Datos recibidos, firma binaria, llave pública, algoritmo.
Valor devuelto	`1` válida, `0` inválida, `-1` error.
Proceso criptográfico aplicado	Descifra la firma con la llave pública, recalcula el hash de los datos y compara ambos resúmenes.

`openssl_encrypt($datos, $metodo, $clave, $opciones, $iv, &$tag)`	
Propósito general	Cifrado simétrico.
Parámetros recibidos	Texto, método (`AES-128-CBC`, `AES-256-GCM`…), clave, opciones (`OPENSSL_RAW_DATA`), IV y tag (solo GCM).
Valor devuelto	Texto cifrado o `false`.
Proceso criptográfico aplicado	AES con bloques de 128 bits. CBC encadena bloques con el IV; GCM usa modo contador y genera un tag de autenticación.

`openssl_decrypt($datos, $metodo, $clave, $opciones, $iv, $tag)`	
Propósito general	Descifrado simétrico.
Parámetros recibidos	Mismos parámetros que `openssl_encrypt` (más el tag en GCM).
Valor devuelto	Texto original o `false`.
Proceso criptográfico aplicado	Operación inversa; en GCM verifica el tag antes de devolver el texto.

`openssl_cipher_iv_length($metodo)`	
Propósito general	Conocer el tamaño del IV de un algoritmo.
Parámetros recibidos	Nombre del método.
Valor devuelto	`int` (16 para AES-CBC, 12 para GCM).
Proceso criptográfico aplicado	No cifra: asegura generar un IV del tamaño correcto.

`openssl_random_pseudo_bytes($largo)`	
Propósito general	Generar bytes aleatorios para IV y sales.
Parámetros recibidos	Número de bytes.
Valor devuelto	Cadena binaria.
Proceso criptográfico aplicado	Generador pseudoaleatorio criptográficamente seguro.

`openssl_public_encrypt() / openssl_private_decrypt()`	
Propósito general	Cifrado asimétrico RSA.
Parámetros recibidos	Datos, salida por referencia, llave, relleno `OPENSSL_PKCS1_OAEP_PADDING`.
Valor devuelto	`bool`.
Proceso criptográfico aplicado	RSA con relleno OAEP: se cifra con la pública y solo la privada descifra; el tamaño máximo depende de la llave.

`openssl_csr_new($dn, &$llave, $config)`	
Propósito general	Crear una solicitud de certificado (CSR).
Parámetros recibidos	Arreglo del nombre distinguido, llave privada, configuración.
Valor devuelto	Recurso CSR o `false`.
Proceso criptográfico aplicado	Empaqueta el nombre y la llave pública y los firma con la llave privada (prueba de posesión).

`openssl_csr_sign($csr, $ca, $llave, $dias, $config)`	
Propósito general	Firmar la CSR y emitir el certificado.
Parámetros recibidos	CSR, certificado de la CA (`null` = autofirmado), llave privada, días de validez, configuración.
Valor devuelto	Recurso X.509 o `false`.
Proceso criptográfico aplicado	El emisor firma el certificado con su llave privada; en un autofirmado el emisor es el propio sujeto.

`openssl_x509_parse()` / `openssl_x509_fingerprint()`	
Propósito general	Leer los campos de un certificado y obtener su huella.
Parámetros recibidos	Certificado y, para la huella, el algoritmo (`sha256`).
Valor devuelto	`array` con sujeto, emisor, serie y vigencia / cadena hexadecimal.
Proceso criptográfico aplicado	Decodifica la estructura X.509 y calcula el hash del certificado en formato DER.

`openssl_error_string()`	
Propósito general	Leer errores de OpenSSL.
Parámetros recibidos	Ninguno.
Valor devuelto	`string` o `false`.
Proceso criptográfico aplicado	Diagnóstico; sirvió para detectar el problema de `openssl.cnf` (sección 7).

5.4 Funciones de Libsodium

`sodium_crypto_secretbox($mensaje, $nonce, $clave)`	
Propósito general	Cifrado simétrico autenticado.
Parámetros recibidos	Mensaje, nonce (24 bytes), clave (32 bytes).
Valor devuelto	Cifrado con MAC de 16 bytes.
Proceso criptográfico aplicado	XSalsa20 para confidencialidad y Poly1305 para integridad en una sola llamada.

`sodium_crypto_secretbox_open($cifrado, $nonce, $clave)`	
Propósito general	Descifrar y verificar.
Parámetros recibidos	Cifrado, nonce, clave.
Valor devuelto	Mensaje o `false` si fue alterado.
Proceso criptográfico aplicado	Verifica el MAC antes de descifrar.

`sodium_crypto_sign_keypair() / sodium_crypto_sign_detached()`	
Propósito general	Crear un par de llaves y firmar.
Parámetros recibidos	Mensaje y llave secreta.
Valor devuelto	Par de llaves / firma de 64 bytes.
Proceso criptográfico aplicado	Ed25519 (curva Edwards25519), firma determinista.

`sodium_crypto_sign_verify_detached($firma, $mensaje, $llavePublica)`	
Propósito general	Verificar una firma Ed25519.
Parámetros recibidos	Firma, mensaje, llave pública.
Valor devuelto	`bool`.
Proceso criptográfico aplicado	Verificación Ed25519.

`sodium_crypto_pwhash_str($clave, $ops, $mem)`	
Propósito general	Hash seguro de contraseñas.
Parámetros recibidos	Contraseña, límites de operaciones y de memoria.
Valor devuelto	Cadena con algoritmo, parámetros, sal y hash.
Proceso criptográfico aplicado	Argon2id: algoritmo de memoria dura que resiste ataques con GPU.

`sodium_crypto_pwhash_str_verify($hash, $clave)`	
Propósito general	Comprobar una contraseña contra su hash.
Parámetros recibidos	Hash guardado, contraseña.
Valor devuelto	`bool`.
Proceso criptográfico aplicado	Recalcula Argon2id con los parámetros contenidos en el hash.

`sodium_crypto_pwhash($largo, $clave, $sal, $ops, $mem, $alg)`	
Propósito general	Derivar una llave desde una contraseña.
Parámetros recibidos	Largo, contraseña, sal de 16 bytes, límites, algoritmo.
Valor devuelto	Llave binaria.
Proceso criptográfico aplicado	Argon2id con sal aleatoria.

`sodium_crypto_box_seal() / sodium_crypto_box_seal_open()`	
Propósito general	Cifrado asimétrico anónimo.
Parámetros recibidos	Mensaje y llave pública (al abrir, el par de llaves).
Valor devuelto	Cifrado / mensaje original.
Proceso criptográfico aplicado	X25519 con llave efímera + XSalsa20-Poly1305; solo el dueño de la llave privada puede abrirlo.

`sodium_memzero(&$variable)`	
Propósito general	Borrar secretos de la memoria.
Parámetros recibidos	Variable por referencia.
Valor devuelto	`void`.
Proceso criptográfico aplicado	Sobrescribe el valor para no dejar llaves en RAM.

6. Investigación y ejemplo práctico con Libsodium (Sodium)

6.1 Investigación teórica

Libsodium es una biblioteca criptográfica moderna, derivada de NaCl y mantenida por Frank Denis, que PHP incluye de forma nativa desde la versión 7.2 (`ext-sodium`). A diferencia de OpenSSL —potente pero amplia, compleja y orientada a protocolos de red y certificados— Sodium se diseñó con enfoque en el desarrollador: hacer que la criptografía moderna sea **segura por defecto y muy difícil de usar mal**. No hay que elegir modos de operación ni rellenos; la biblioteca selecciona algoritmos robustos y entrega siempre cifrado autenticado.

Principales algoritmos

Operación	Algoritmo	Funciones PHP
Cifrado simétrico autenticado	XSalsa20-Poly1305 / XChaCha20-Poly1305	`sodium_crypto_secretbox`, `sodium_crypto_aead_xchacha20poly1305_ietf_encrypt`
Cifrado asimétrico	X25519 + XSalsa20-Poly1305	`sodium_crypto_box`, `sodium_crypto_box_seal`
Firmas digitales	Ed25519	`sodium_crypto_sign_*`
Hash de contraseñas	Argon2id (recomendado por OWASP)	`sodium_crypto_pwhash_str`
Hash genérico	BLAKE2b	`sodium_crypto_generichash`

Ventajas de seguridad

- **Resistente al mal uso:** gestiona nonces, aleatoriedad y autenticación, evitando errores comunes como repetir un IV o cifrar sin verificar integridad.
- **Argon2id** para contraseñas: memoria dura, resiste la fuerza bruta con GPU.
- **Cifrado autenticado** en una sola llamada (confidencialidad e integridad).
- **Operaciones en tiempo constante** y borrado de secretos (`sodium_memzero`).
- **Cero dependencias extra:** viene incluida en WampServer y XAMPP, sin archivos `openssl.cnf`.

Ventajas de rendimiento (medidas en este laboratorio, captura 32)

Operación	OpenSSL	Sodium	Observación
Generar par de llaves	RSA-2048: 58.020 ms	Ed25519: 0.064 ms	Sodium ≈ 900 veces más rápido

Operación	OpenSSL	Sodium	Observación
Firmar un mensaje	RSA-2048 + SHA-256: 0.440 ms	Ed25519: 0.079 ms	Sodium ≈ 5.6 veces más rápido
Cifrar 1 MB	AES-256-GCM: 0.201 ms	secretbox: 1.689 ms	AES-GCM de OpenSSL es más rápido porque el procesador lo acelera por hardware (AES-NI)
Tamaño de llave pública / firma	294 B / 256 B	32 B / 64 B	Ed25519 usa llaves y firmas mucho más pequeñas

La conclusión es matizada: Sodium gana claramente en llaves y firmas, mientras que para cifrar grandes volúmenes con AES-GCM OpenSSL aprovecha el hardware. La ventaja principal de Sodium es la seguridad y simplicidad, no siempre la velocidad.

OpenSSL frente a Sodium: ¿cuándo usar cada uno?

Característica	OpenSSL	Libsodium (ext-sodium)
Propósito principal	Protocolos de red, certificados digitales y criptografía asimétrica (RSA).	Criptografía moderna simétrica, firmas y derivación/hashing de alta seguridad.
Complejidad	Alta: múltiples parámetros (modos, rellenos, longitudes, IV).	Baja: funciones simples y seguras desde la primera línea.
Uso recomendado	Conexiones HTTPS, llaves públicas/privadas y certificados.	Contraseñas y cifrado de datos sensibles en la base de datos.

6.2 Ejemplo implementado (sodium.php)

El script `sodium.php` integra cinco demostraciones con formulario (figuras 21 a 32): **(A)** cifrado autenticado con contraseña (Argon2id + secretbox), **(B)** firma digital Ed25519, **(C)** almacenamiento de contraseñas con Argon2id, **(D)** sealed box y **(E)** comparativa de rendimiento contra OpenSSL. Núcleo del ejemplo de cifrado autenticado:

```
// Derivar una llave de 32 bytes desde la contraseña (Argon2id)
$sal   = random_bytes(SODIUM_CRYPTO_PWHASH_SALTBYTES);
$clave = sodium_crypto_pwhash(
    SODIUM_CRYPTO_SECRETBOX_KEYBYTES, $contrasena, $sal,
    SODIUM_CRYPTO_PWHASH_OPSLIMIT_INTERACTIVE,
    SODIUM_CRYPTO_PWHASH_MEMLIMIT_INTERACTIVE,
    SODIUM_CRYPTO_PWHASH_ALG_ARGON2ID13
);

// Cifrar y autenticar en una sola llamada
$nonce   = random_bytes(SODIUM_CRYPTO_SECRETBOX_NONCEBYTES);
$cifrado = sodium_crypto_secretbox($mensaje, $nonce, $clave);

// Descifrar: devuelve false si el mensaje fue alterado o la clave es incorrecta
$abierto = sodium_crypto_secretbox_open($cifrado, $nonce, $clave);
if ($abierto === false) { echo 'Mensaje alterado o clave incorrecta'; }
sodium_memzero($clave);
```

Resultado observado: con la contraseña correcta se recupera el mensaje (figura 23); si se altera el cifrado, `secretbox_open` devuelve `false` (figura 24); la firma Ed25519 se verifica solo con el mensaje original (figuras 26 y 27); y la contraseña errónea es rechazada por Argon2id (figura 30). Así se protege la confidencialidad y la integridad, como pide OWASP.

7. Dificultades encontradas y soluciones

#	Dificultad	Solución aplicada
1	`firma7.php` y `firmaOtra.php` originales fallaban: `firmaOtra` apunta a `C:\OpenSSL-Win64\bin\cnf\openssl.cnf`, ruta inexistente en WampServer (figuras 33 y 34).	Se aplicaron las Soluciones 1 y 2 de la guía en `includes/bootstrap.php`: `localizarOpensslCnf()` busca el archivo en `OPENSSL_CONF`, en la carpeta de PHP de WampServer, en XAMPP (cualquier unidad) y en OpenSSL independiente; si no lo halla, omite el parámetro `config`.
2	`FirmarMensaje.php` original tomaba la llave pública de `certout.csr`, que es una solicitud y no un certificado.	`firmaOtra.php` ahora emite `certout.crt` y `FirmarMensaje.php` extrae la llave pública de ese certificado.
3	`DiprenaGuardarAprobacion.php` no se ejecuta fuera de su sistema: requiere `Barricada.php`, `clases/`… y funciones `mysql_*` (eliminadas en PHP 7).	Su parte criptográfica se reprodujo en `firmaDiprena.php` sin base de datos; el original se conserva en `originales/`.
4	Los scripts originales usaban datos fijos en el código y no tenían formularios.	Se agregaron formularios con token CSRF, escape de salida y botones para simular alteraciones.
5	Riesgo de publicar llaves privadas en GitHub.	`.gitignore` excluye `keys/*` y `keysCert/*`; los `.htaccess` impiden su descarga por HTTP.

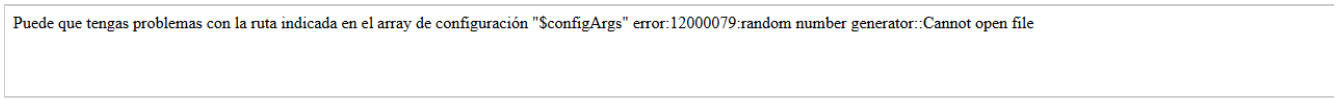


Figura 33. Error del script original firma7 (sin openssl.cnf)

> Con la ruta de configuración del original, OpenSSL responde `error:12000079:random number generator::Cannot open file` y no genera el par de llaves.



Figura 34. Error del script original firmaOtra (ruta de openssl.cnf)

> El script original responde `error:10000080:BIO routines::no such file` porque la ruta fija a `C:\OpenSSL-Win64` no existe en este equipo; con `localizarOpensslCnf()` el script funciona (figura 13).

8. Conclusiones personales

Este laboratorio me mostró que la criptografía en PHP se reduce a pocas funciones, pero un detalle mal configurado la deja sin protección: un IV repetido, una clave escrita en el código o un cifrado sin verificación de integridad. Al alterar los datos en los formularios comprobé la diferencia entre **cifrar** (confidencialidad) y **autenticar** (integridad): AES-CBC sin MAC puede descifrar basura sin avisar, mientras que AES-GCM, HMAC y `secretbox` rechazan el mensaje modificado.

También entendí el papel de cada pieza de la firma digital: la llave privada firma, la pública verifica y el certificado X.509 asocia esa llave a una identidad. Resolver la ausencia de `openssl.cnf` y la confusión entre CSR y certificado me enseñó a leer los errores con `openssl_error_string()` y a no depender de rutas fijas, lo que coincide con la proactividad ante errores que valora la guía.

Con Libsodium vi que hacer lo correcto puede ser más sencillo: menos parámetros, nonce y MAC integrados, y Argon2id para contraseñas. La medición de rendimiento me dejó una lección crítica: Sodium no siempre es más rápido (AES-256-GCM de OpenSSL ganó al cifrar 1 MB gracias al hardware), pero sí es más difícil de usar mal. Mi conclusión es usar **OpenSSL cuando se necesitan certificados y compatibilidad** y **Sodium para cifrar datos sensibles y guardar contraseñas**, aplicando los principios de A02 de OWASP y evitando publicar llaves privadas en el repositorio.

9. Referencias

1. Fong, I. (2026). *Guía para la entrega del laboratorio: Seguridad en PHP con OpenSSL*. Universidad Tecnológica de Panamá.
2. Salomon2514. *SeguridadOpenSSL* [Repositorio]. GitHub. Scripts originales del laboratorio.
3. PHP Group. *OpenSSL – Manual de PHP*. https://www.php.net/manual/es/book.openssl.php
4. PHP Group. *Sodium – Manual de PHP*. https://www.php.net/manual/es/book.sodium.php
5. Libsodium. *Documentación de libsodium*. https://doc.libsodium.org/
6. OWASP. (2021). *A02:2021 – Cryptographic Failures*. https://owasp.org/Top10/A02_2021-Cryptographic_Failures/
7. OWASP. *Password Storage Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
8. OpenSSL Project. *OpenSSL documentation*. https://docs.openssl.org/